

基于时空网络、随机动态规划与拥挤博弈的穿越沙漠策略研究

摘 要

本文研究有限资金、承重与天气约束下的沙漠穿越问题。六关的核心差异并非地图大小，而是信息集由“未来天气全知”依次变化为“仅知当天气象”和“可观察其他玩家历史”。据此，本文建立“时间扩展网络混合整数规划—有限时域马尔可夫决策过程—多人动态博弈”的递进框架，并用独立规则模拟器、全序列枚举、预先规定风险预算和最佳响应检验形成分级证据链。

针对问题一，将位置流、逐日行动、资源库存、现金、负重、村庄补给与矿山收益统一写入时间扩展网络 MILP。HiGHS 对第一、二关均返回全局最优，独立模拟器逐日回放后，第一关以水 178 箱、食物 333 箱出发，第 24 天到达，最终资金为 **10470 元**；第二关以水 130 箱、食物 405 箱出发，第 30 天到达，最终资金为 **12730 元**。回放还发现公开参考表第二关第 15–18 天水量少扣 10 箱，但修正后最优值不变，说明“数值最优”和“逐日合法”必须分别审计。

针对问题二，题面未给未来天气分布，故最佳策略只能是条件结论。第三关在晴朗/高温独立分布下建立天气先揭示的精确有限时域 MDP；高温概率为 0.2、0.5、0.8 时，最优初始装载均为水 54 箱、食物 54 箱，沿 $1 \rightarrow 4 \rightarrow 6 \rightarrow 13$ 三日直达，期望终值分别为 9382、9310、9238 元，并对每档全部 $2^{10} = 1024$ 条天气序列实现零失败。第四关先在 IID 下从 567 个装载—沙暴预算候选选出 240/240 箱、B5 效率策略；再构造 $P_\rho = \rho I + (1 - \rho)\mathbf{1}p^\top$ ，在保持天气边际概率不变的条件下把持续性扩展到 $\rho \leq 0.35$ 。分布鲁棒重选得到 240/240 箱、B9 策略；在最不利边际、 $\rho = 0.35$ 的 30000 条样本中，失败率为 0.1967% (Wilson 上界 0.2536%)，而 IID-B5 为 2.2300%。

针对问题三，首先从原始 Word 公式核实同边 k 人移动的消耗系数为 $2k$ 、同矿收益为基础收益的 $1/k$ 。第五关对固定对手计划执行完整最佳响应动态规划，得到终值 **9535 元** 和 **9510 元** 的非对称纯策略 Nash 均衡，双方最大单边偏离增益均为 0。第六关实现三人联合反馈模拟；机械复制单人挖矿策略在 IID 基准天气下玩家失败率高达 97.865%。进一步按角色校准二维装载，最终采用 215/215、215/215 与 225/230 箱的异路错峰剖面；在最不利边际、 $\rho = 0.35$ 下玩家失败率 0.3400%、固定角色最大 Wilson 上界 0.4344%，总体均值 **7383.3 元**。受约束最佳响应预言机对每名玩家完整检查 70 条最短路乘 2 种风险可行错峰装载，共 140 个偏离，样本外最大可利用度为 **0 元**。公开随机分配三种角色实现事前公平。

本文的创新点在于：以信息结构而非题号选择模型；将失败样本按 0 元纳入总体效用以消除幸存者偏差；用保持边际不变的 Markov 模型分离天气频率与时间聚集；将“全局最优、精确枚举、均衡偏离与分布鲁棒样本外推荐”严格分级。 $\rho = 0.70$ 的集合外破坏性测试明确展示保护边界，因而不把条件安全包装成任意天气下的无条件最优。

关键词：时间扩展网络；混合整数规划；马尔可夫决策过程；鲁棒策略；Nash 均衡；拥挤外部性；样本外验证

1 问题重述与分析

1.1 问题的统一结构

玩家在图 ($G=(V,E)$) 上从起点走向终点。每天只能移动到相邻区域或停留；沙暴日不能移动；在矿山可用更高消耗换取收入，在村庄可用更高价格补给。水、食物、现金和负重共同限制可行性，到达终点后剩余物资按基准价一半回收。六关共享同一物理系统，但给定信息和玩家数量不同：

- 问题一：未来天气完全已知，是确定性、有限时域的网络优化；
- 问题二：行动前只知当天天气，是满足非前视约束的反馈控制；
- 问题三：收益和消耗还依赖他人同日行动，是带拥挤外部性的动态博弈。

从确定性最优到多人反馈：统一状态—决策—证据链



证据等级随信息条件变化：不把样本外推荐误称为无条件全局最优

图 1 由信息集决定的统一模型与验证链

1.2 四个建模难点

难点一是库存和位置强耦合。同一条几何最短路未必经济最优：晴朗、高温和沙暴的资源成本不同，绕行矿山可能增加现金，也可能因额外移动和负重失去可行性。

难点二是信息时序。第三、四、六关只能依据日期、当前状态、当天气象和已观测历史行动，不得使用未来天气。若先生成整条天气再让策略读取，就是信息泄漏。

难点三是“较少沙暴”并非概率。题面没有唯一概率分布，因而不存在脱离假设的唯一期望最优策略。本文把概率视为建模参数，分别报告低沙暴、基准和不利分布，并用风险预算筛选策略。

难点四是多人规则带来负外部性。三人复制同一单人策略会在同一有向边形成 (2k) 倍消耗，在矿山又分摊收益。问题三必须同时分析路线错峰、矿山竞争、角色公平和单边偏离。

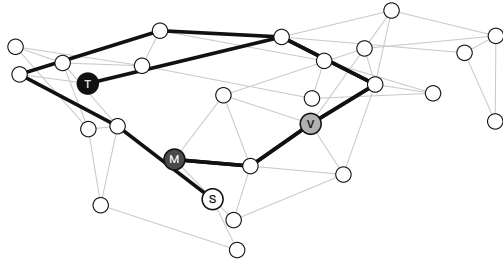
1.3 地图规范化

附件地图被重建为无向邻接表,并自动检查节点完备、邻接对称和全图连通。第三/五关、第四/六关分别共享同一图,因此六关实际只涉及四种图结构,见图 2。图中第一、二关的粗黑线是求解器输出的最优轨迹,不是人工描线。

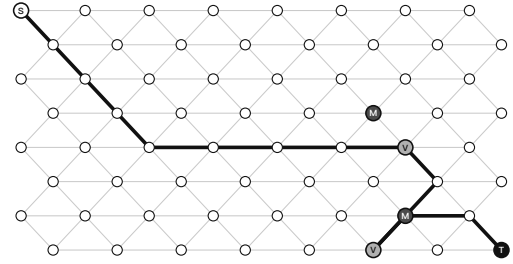
六关地图归并为四种图结构

S起点 · T终点 · V村庄 · M矿山; 粗黑线为第一、二关最优轨迹

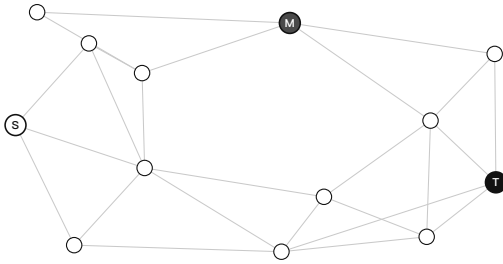
第一关：27区不规则图



第二关：8x8六边形图



第三/五关：13区图



第四/六关：5x5网格

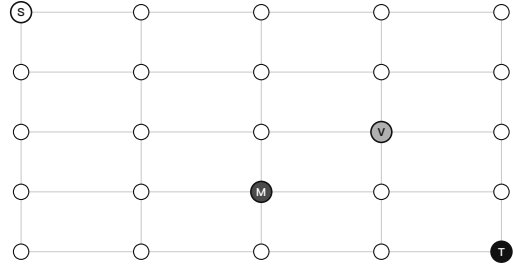


图 2 六关地图的规范图结构与确定天气最优轨迹

2 统一建模框架

2.1 符号、决策时序与基本假设

表 1 核心符号与含义

符号	含义	符号	含义
$G = (V, E)$	区域邻接图	T	截止日期
$z_t \in \mathcal{Z}$	第 t 天天气	l_t	第 t 日日末位置
w_t, f_t	日末水、食物库存	m_t	日末现金
a_t	移动、停留或挖矿行动	$b_t^{r,-}, b_t^{r,+}$	行动前、后购买量
c_z^r	天气 z 下资源 r 的基础消耗	λ_t	行动消耗倍数
B	矿山基础收益	C	负重上限
τ	首次到达终点的日期	$\mathcal{F}_t$	第 t 日可用信息集
$k_{t,e}, k_{t,v}$	同边移动、同村购买人数	J_i	玩家 i 的终值

本文把每一天严格拆成“天气揭示—行动前采购—行动—资源消耗与收益结算—行动后采购”五步。记上日日末状态为

$$S_{t-1} = (l_{t-1}, w_{t-1}, f_{t-1}, m_{t-1}), \quad (1)$$

行动前采购后的临时库存为

$$\tilde{r}_{t-1} = r_{t-1} + b_t^{r,-}, \quad r \in \{w, f\}. \quad (2)$$

观察当天气象 z_t 后选择 a_t ，消耗后、行动后采购前的库存为

$$\bar{r}_t = \tilde{r}_{t-1} - \lambda(a_t)c_{z_t}^r, \quad \lambda(a_t) = \begin{cases} 2, & a_t = \text{移动}, \\ 1, & a_t = \text{停留}, \\ 3, & a_t = \text{挖矿}. \end{cases} \quad (3)$$

最后有 $r_t = \bar{r}_t + b_t^{r,+}$ 。这一区分很重要：到达村庄当天只能用出发前已有物资完成移动，不能用“尚未到达的村庄补给”倒推当日可行性。因此模型同时要求

$$\bar{w}_t \geq 0, \quad \bar{f}_t \geq 0, \quad 3\tilde{w}_{t-1} + 2\tilde{f}_{t-1} \leq C, \quad 3w_t + 2f_t \leq C. \quad (4)$$

为使六关可以在同一框架内比较，作如下最小假设：资源与现金均按题面单位离散；玩家同日共享同一天气；只有位于村庄时可以购买，初始购买按基准价、村庄购买按题面倍率；到达终点后进入零消耗吸收状态；未在截止日前到达或中途资源不可行为失败。除第二、三问的条件概率模型外，不额外假定未来天气可见。

2.2 统一状态转移与终值

设 κ_t 为当日采购价格倍率， I_t^M 为挖矿指示变量，则单人现金转移为

$$m_t = m_{t-1} + BI_t^M - \sum_{r \in \{w, f\}} p_r (\kappa_t^- b_t^{r,-} + \kappa_t^+ b_t^{r,+}). \quad (5)$$

位置由映射 L 更新： $l_t = L(l_{t-1}, a_t)$ 。将式 (2)、(3) 和 (5) 合并，记确定性日转移为

$$S_t = \Phi(S_{t-1}, z_t, a_t, \mathbf{b}_t^-, \mathbf{b}_t^+). \quad (6)$$

若 $\tau = \inf\{t : l_t = v_T\} \leq T$ ，题面终值为

$$J = m_\tau + \frac{1}{2}p_w w_\tau + \frac{1}{2}p_f f_\tau; \quad (7)$$

若失败则令 $J = 0$ 。因此随机天气下真正用于选模的目标是总体效用

$$U(\pi) = \mathbb{E}_\pi[J] = \Pr_\pi(\tau \leq T) \mathbb{E}_\pi[J \mid \tau \leq T], \quad (8)$$

而不是只统计成功者的条件均值。式 (8) 也解释了为何高收益但高失败率的挖矿策略会被风险约束淘汰。

2.3 信息结构与可行策略集

确定天气关卡中 $z_{1:T}$ 在第 0 日已知；在线关卡中定义

$$\mathcal{F}_t = \sigma(z_1, \dots, z_t, S_0, \dots, S_{t-1}), \quad (9)$$

并要求策略 π_t 对 $\mathcal{F}_t$ 可测，即

$$(a_t, \mathbf{b}_t^-, \mathbf{b}_t^+) = \pi_t(\mathcal{F}_t), \quad \pi_t \text{ 不得依赖 } z_{t+1:T}. \quad (10)$$

多人情形把其他玩家的历史位置、公开行动和共同天气加入 $\mathcal{F}_t$ ，但仍不允许读取未来。对任一状态和天气，合法行动集记为 $\mathcal{A}(S_{t-1}, z_t)$ ；它同时编码邻接、沙暴禁行、矿山位置、采购位置、非负库存、现金与负重限制。于是六关的差异被压缩为三件事：天气信息量、参与者数量，以及求解 $\max_{\pi \in \Pi} U(\pi)$ 时采用的证据强度。

2.4 证据等级与统一验收

为避免把不同强度的计算结论混在一起，本文采用四级证据：

1. **[E1]** 求解器全局最优证书与独立回放，对应“全局最优”；
2. **[E2]** 完整状态空间和天气支撑精确枚举，对应“给定模型下精确最优”；
3. **[E3]** 完整最佳响应检验，对应“给定策略空间内的纯策略均衡”；
4. **[E4]** 独立种子样本外评估、置信区间与敏感性分析，对应“条件推荐”。

每个结果都经过与求解程序解耦的规则入口回放；每个随机策略都同时报告失败率、Wilson 区间、含失败总体均值及其区间。正文只引用通过验收的图表，候选图与中间产物保留在独立输出目录，不进入最终论文。

统一边界

问题一可以证明全局最优；第三、五关可以在给定有限支撑或策略空间内精确验证；第四、六关因题面未给天气分布，只能在声明分布和预设风险预算下推荐。四类表述不可互换。

3 问题一：确定天气下的时空网络优化

3.1 时间扩展网络 MILP

3.1.1 行动弧与位置流

对每个日期 t 构造合法行动弧集 $\mathcal{A}_t$ 。一条弧 $\alpha \in \mathcal{A}_t$ 包含起点 $o(\alpha)$ 、终点 $d(\alpha)$ 和行动类型 $g(\alpha) \in \{W, S, M, H\}$ ，依次表示移动、停留、挖矿和到达后的吸收自环。沙暴日不生成移动弧，非矿山不生成挖矿弧，终点只生成零消耗吸收弧。定义

$$x_{ti} = \mathbb{I}\{\text{第 } t \text{ 日日末位于 } i\}, \quad \xi_{t\alpha} = \mathbb{I}\{\text{第 } t \text{ 日选择弧 } \alpha\}.$$

起点条件和每日流守恒为

$$x_{0i} = \mathbb{I}\{i = v_S\}, \quad (11)$$

$$\sum_{\alpha: o(\alpha)=i} \xi_{t\alpha} = x_{t-1,i}, \quad i \in V, \quad t = 1, \dots, T, \quad (12)$$

$$\sum_{\alpha: d(\alpha)=j} \xi_{t\alpha} = x_{tj}, \quad j \in V, \quad t = 1, \dots, T, \quad (13)$$

$$x_{T,v_T} = 1. \quad (14)$$

由 $\sum_i x_{0i} = 1$ 和式 (12) 和 (13) 可归纳得到每天恰选一条行动弧；到达矿山当天的弧起点仍是上一日位置，故不可能“到达即挖矿”。吸收弧使提前到达与统一终止日 T 兼容，而且到达后不再消耗资源。

3.1.2 资源、采购与现金约束

记弧 α 在第 t 日对资源 r 的消耗为

$$d_{t\alpha}^r = c_{z_t}^r \begin{cases} 2, & g(\alpha) = W, \\ 1, & g(\alpha) = S, \\ 3, & g(\alpha) = M, \\ 0, & g(\alpha) = H. \end{cases} \quad (15)$$

初购变量为 b_0^r ，到达村庄 v 后的采购变量为 b_{tv}^r 。资源平衡、采购位置和“采购前亦不得为负”分别写成

$$r_0 = b_0^r, \quad (16)$$

$$r_t - r_{t-1} - \sum_{v \in V_V} b_{tv}^r + \sum_{\alpha \in \mathcal{A}_t} d_{t\alpha}^r \xi_{t\alpha} = 0, \quad (17)$$

$$0 \leq b_{tv}^r \leq M_r x_{tv}, \quad (18)$$

$$r_t - \sum_{v \in V_V} b_{tv}^r \geq 0. \quad (19)$$

其中 $M_w = \lfloor C/3 \rfloor$ 、 $M_f = \lfloor C/2 \rfloor$ 已足够，不使用任意大的常数。负重在初购与每日采购后均检查：

$$3w_t + 2f_t \leq C, \quad t = 0, \dots, T. \quad (20)$$

设 $I_{t\alpha}^M = 1$ 表示弧 α 为挖矿弧，则现金平衡为

$$m_0 + p_w b_0^w + p_f b_0^f = M_0, \quad (21)$$

$$m_t - m_{t-1} + 2p_w \sum_v b_{tv}^w + 2p_f \sum_v b_{tv}^f - B \sum_{\alpha \in \mathcal{A}_t} I_{t\alpha}^M \xi_{t\alpha} = 0. \quad (22)$$

所有库存、采购量和现金均为非负整数， $x_{ti}, \xi_{t\alpha}$ 为二元变量。为避免半价回收产生小数，实际求解最大化二倍终值

$$\max Z = 2m_T + p_w w_T + p_f f_T, \quad J^* = \frac{Z^*}{2}. \quad (23)$$

3.1.3 模型等价性与复杂度

命题 1：满足式 (11)和(23)的任一整数解，与一条合法的逐日穿越计划一一对应。

证明要点：由起点条件和两组流平衡，可逐日唯一恢复所选行动弧； $\mathcal{A}_t$ 的构造已排除沙暴移动、非邻接移动和非矿山挖矿。采购连接约束只允许在实际到达的村庄买入；式 (19)排除了用行动后采购支付行动前消耗；资源、现金和负重方程逐日守恒；终点条件保证按期到达。反之，任一合法计划把每天对应弧置 1、按账本赋值库存和现金，即满足全部约束。故两者双射，目标式 (23)与题面终值完全一致。□

模型含 $O(T|V|)$ 个位置变量、 $O(T(|E| + |V|))$ 个行动变量和 $O(T|V_V|)$ 个采购变量，约束数量同阶。其网络流骨架稀疏，只有采购、资源和现金方程把各层耦合，因此可由分支定界结合线性松弛有效求解 [2]。

3.2 求解、回放与最优性证书

模型由 HiGHS 1.15.1 求解。HiGHS 的学术引用依据其官方说明采用 Huangfu 与 Hall 的并行对偶修正单纯形论文 [6]。求解器目标值返回后，不直接把变量表作为答案，而是将其转换成逐日计划，交由独立规则模拟器重新执行以下检查：邻接、沙暴移动、到矿当日挖矿、购买位置、采购现金、负重、资源非负和到达后停止。

表 2 第一、二关全局最优解摘要

关卡	初始水/食物	最优终值/元	到达日	求解状态	路线与关键行为
第一关	178/333	10470	24	Optimal	经 15 村庄、12 矿山，挖矿 7 天，两次村庄补给
第二关	130/405	12730	30	Optimal	经 39、62 村庄和 55 矿山，累计挖矿 12 天

3.3 第一关策略解释

最优路径为

$$1 \rightarrow 25 \rightarrow 26 \rightarrow 23 \rightarrow 21 \rightarrow 9 \rightarrow 15 \rightarrow 13 \rightarrow 12,$$

在第 11–16、19 日于 12 区挖矿；随后经 $13 \rightarrow 15 \rightarrow 9 \rightarrow 21 \rightarrow 27$ 到达。第 8 日到 15 村庄后购买 163 箱水，第 21 日再次到 15 村庄购买 36 箱水和 16 箱食物。第一段采购主要修正水相对食物的消耗不平衡；第二段补给把终点剩余压到 0，从而避免以全价或双倍价买入后只按半价回收。

矿山路线比最短路更长，但 7 天基础收益共 7000 元，足以覆盖绕行、挖矿三倍消耗和村庄溢价。求解器自动选择在沙暴第 11 日挖矿，却在连续沙暴第 17、18 日停留；原因不是沙暴不能挖矿，而是后两日继续挖矿会使返程资源和补给现金条件恶化。

3.4 第二关策略解释与数据审计

第二关先经 $1 \rightarrow 9 \rightarrow 18 \rightarrow 26 \rightarrow 35 \rightarrow 36 \rightarrow 37 \rightarrow 38 \rightarrow 39$ ，在 39 村庄分两日补水；再经 47 到 55 矿山。第 14–18 日挖矿，第 19 日到 62 村庄补给 201 箱水和 86 箱食物，第 20 日返回 55 矿山，第 21–28 日中继续挖矿 7 天，最后经 56 到 64。

原公开参考轨迹把第 15–18 日水量依次记为 110、86、56、26 箱；按高温/沙暴挖矿三倍消耗应为 100、76、46、16 箱。第 19 日先完成高温移动，再购买 201 箱水和 86 箱食物，日末水量为 201 箱，与现金减少

$$201 \times 2 \times 5 + 86 \times 2 \times 10 = 3730$$

完全一致。修正后终值仍为 12730 元，且求解器找到不同路线但同目标值的另一最优解，说明本关存在多重最优。

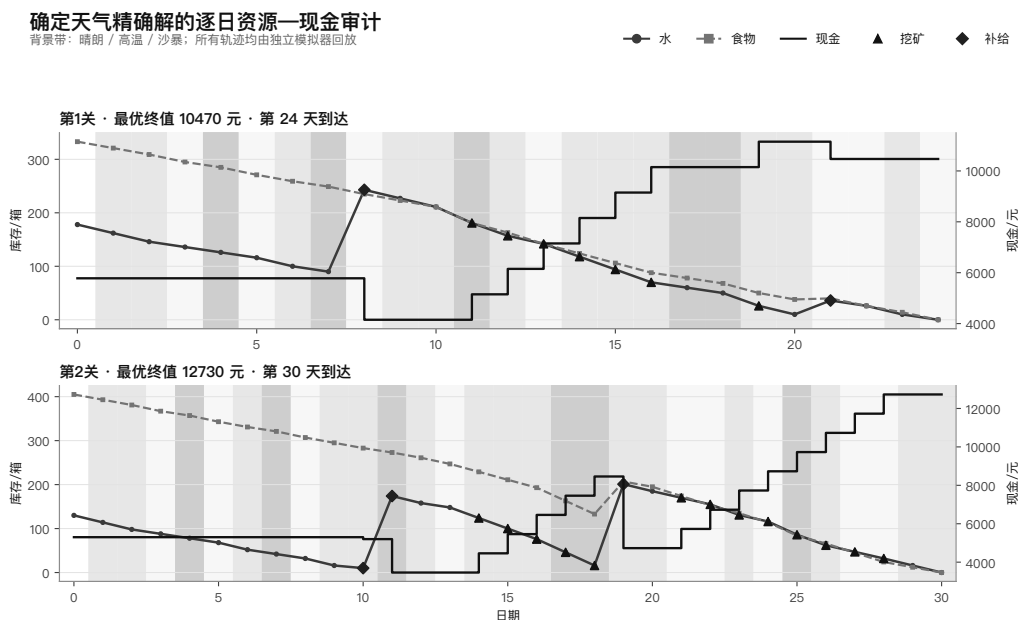


图 3 第一、二关最优策略的逐日库存、现金、天气与关键行动

问题一结论 [E1]

第一关最优终值 10470 元、第二关 12730 元，均由求解器全局最优状态和独立逐日回放共同支持。图3同时证明补给、挖矿、现金与库存能够闭合，而非只有最终数值正确。

3.5 一般情况下的最优策略

对任意地图、确定天气和有限截止日，只需替换邻接表、特殊节点与参数，即可复用上述时间扩展网络。若规模扩大，可先用终点和矿山的最短距离做可达性剪枝，再对采购量建立更紧的上下界；若所有资源和现金保持整数，可直接使用分支定界给出最优间隙。故“一般策略”不是固定路线，而是：在完整天气路径下解联合位置—库存—现金 MILP，再用独立模拟器验证。

4 问题二：仅知当天天气的反馈决策

4.1 信息约束与概率边界

设 $H_t = (S_0, z_1, a_1, \dots, S_{t-1}, z_t)$ 为行动前完整历史， $\mathcal{F}_t = \sigma(H_t)$ 。在线策略 $\pi_t(H_t)$ 必须满足式 (10)；代码层面只向策略对象传入历史记录和当天气象，未来数组封装在外部环境中，因而信息约束不是文字承诺，而是接口约束。天气概率并非题面事实，本文作如下条件设定：第三关无沙暴，以高温概率 $p_h \in \{0.2, 0.5, 0.8\}$ 的独立分布做精确敏感性；第四关分别使用

$$(p_s, p_h, p_b) = (0.50, 0.45, 0.05), (0.45, 0.45, 0.10), (0.35, 0.50, 0.15)$$

代表低沙暴、基准和不利天气。

4.2 第三关有限时域精确 MDP

第三关没有村庄，现金只受初始采购、挖矿和终点回收影响。给定初购后，历史对未来的作用可由充分状态 $s = (l, w, f)$ 概括。为准确体现“先观测天气再行动”，分别定义天气揭示前价值 $V_t(s)$ 和揭示后价值 $Q_t(s, z)$ ：

$$Q_t(s, z) = \max_{a \in \mathcal{A}(s, z)} \{R(s, a, z) + V_{t+1}(F(s, a, z))\}, \quad (24)$$

$$V_t(s) = \sum_{z \in \mathcal{Z}} p_z Q_t(s, z). \quad (25)$$

其中即时收益 $R = B$ 当且仅当挖矿，否则为 0； F 按式 (3) 扣减资源并更新位置。边界条件为

$$V_t(v_T, w, f) = \frac{1}{2}p_w w + \frac{1}{2}p_f f, \quad V_{T+1}(l, w, f) = -\infty \quad (l \neq v_T). \quad (26)$$

现金在途中不影响行动可行性，故可从状态中剔除，并在初购层补回。完整优化为

$$\max_{w_0, f_0 \in \mathbb{Z}_+} \{M_0 - p_w w_0 - p_f f_0 + V_1(v_S, w_0, f_0)\} \quad \text{s.t.} \quad 3w_0 + 2f_0 \leq C. \quad (27)$$

若某个 $p_z > 0$ 而 $Q_t(s, z) = -\infty$ ，则令 $V_t(s) = -\infty$ 。这一“支撑集稳健可行”递推排除了以小概率必败换取较高条件均值的方案。

有限时域和整数资源使状态空间有限。采用记忆化递归后，每个 (t, l, w, f) 只计算一次；同一状态、同一资源消耗下仅保留收益最大的行动。最坏复杂度为 $O(T|V|WF|\mathcal{Z}|\mathcal{A}|)$ ，其中 W, F 分别是容量给出的水、食物整数上界。

程序枚举全部整数初始采购和可达状态。每档高温概率评估 6425464 个状态，并对全部 $2^{10} = 1024$ 条天气序列逐条回放，Bellman 期望与枚举加权均值误差小于 10^{-6} 。

表 3 第三关精确 MDP 对高温概率的敏感性

高温概率	初始水/食物	期望终值	5% 分位	最坏值	失败序列
0.2	54/54	9382	9270	9190	0/1024
0.5	54/54	9310	9190	9190	0/1024
0.8	54/54	9238	9190	9190	0/1024

三档概率下策略均为 $1 \rightarrow 4 \rightarrow 6 \rightarrow 13$ 连续移动三天。全高温路径的最低资源由

$$w_0^{\min} = 3 \times 2c_{\text{高温}}^w = 54, \quad f_0^{\min} = 3 \times 2c_{\text{高温}}^f = 54 \quad (28)$$

给出，恰好等于 MDP 解。绕矿需要五个移动日，矿山基础收益仅 200 元，额外稳健装载和绕行成本超过挖矿收益。因此“54/54 直达”既有 Bellman 最优性，也有由最坏支撑资源下界给出的结构解释。

第三关：高温概率升高只压低价值，不改变54/54直达策略

每个概率点精确枚举1024条天气序列，失败数均为0

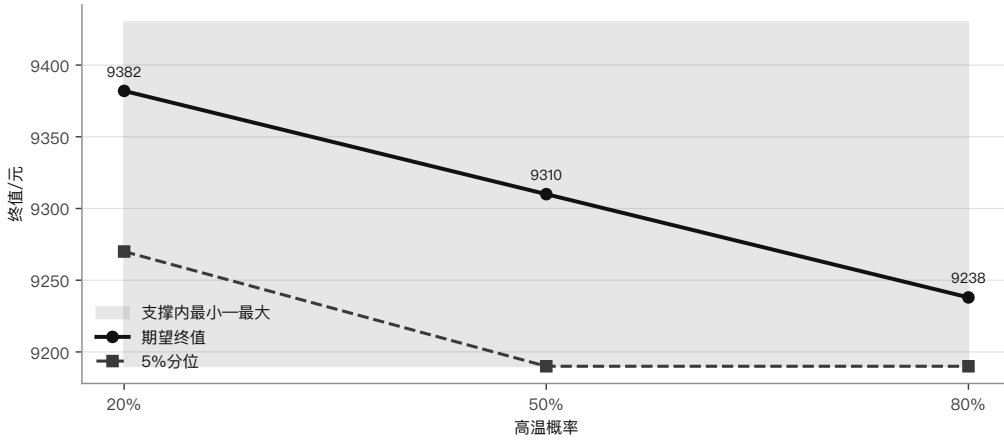


图 4 第三关高温概率、期望终值与支撑内风险

4.3 第四关鲁棒阈值策略

第四关状态空间远大于第三关，且含沙暴、矿山和较长时域，直接精确 DP 在多档参数搜索中代价过高。我们从“挖矿后仍必须保证返程”的充分条件出发构造可解释阈值策略。设 $d_t = d(l_t, v_T)$ 为当前位置到终点最短边数， $N_b(t)$ 为截至当天已观察到的沙暴数， B 为设计的总沙暴预算，剩余预算为

$$\hat{B}_t = (B - N_b(t))_+. \quad (29)$$

若以后最多再遇 $\hat{B}_t$ 个沙暴，则 d_t 个移动日按高温上界需 $2d_t c_h^r$ ，沙暴停留需 $\hat{B}_t c_b^r$ 。因此第 t 日挖矿后保持返程可行的充分条件为

$$T - t \geq d_t + \hat{B}_t, \quad (30)$$

$$w_{t-1} - 3c_{z_t}^w \geq 2d_t c_{\text{高温}}^w + \hat{B}_t c_{\text{沙暴}}^w, \quad (31)$$

$$f_{t-1} - 3c_{z_t}^f \geq 2d_t c_{\text{高温}}^f + \hat{B}_t c_{\text{沙暴}}^f. \quad (32)$$

式中右端第一项的系数 2 来自移动消耗倍数，第二项的系数 1 来自沙暴停留。三式任一不满足时，非沙暴日立即沿最短路向终点移动，沙暴日停留。阈值不是对所有天气路径的绝对保证，而是把风险偏好压缩到一个可搜索、可解释的整数 B 。

4.4 567 候选选模与独立测试

枚举 $B = 3, \dots, 9$ ，初始水食 180–280 箱、步长 10，并过滤负重与现金不合法组合，得到 567 个候选 Θ 。对每个 $\theta \in \Theta$ ，在基准和不利分布各使用 1000 条相同的配对天气序列，减少候选比较中的抽样噪声。训练选择规则在模拟前固定为

$$\hat{\theta} = \arg \max_{\theta \in \Theta} \hat{\mu}_{\text{adv}}(\theta) \quad \text{s.t.} \quad \hat{p}_{\text{base}}(\theta) \leq 0.002, \quad \hat{p}_{\text{adv}}(\theta) \leq 0.005, \quad (33)$$

其中 $\hat{\mu}$ 按式 (8) 把失败样本记 0。入选水 240 箱、食物 240 箱、 $B = 5$ 。图 5 显示，入选点不是训练均值最大的点，而是风险预算内的收益折中。

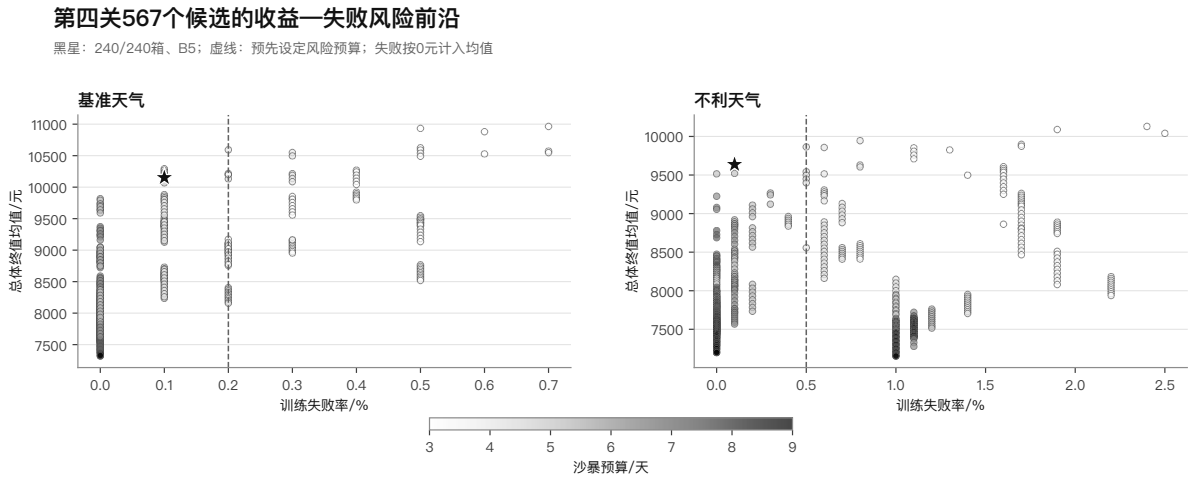


图 5 第四关候选策略在基准和不利天气下的训练收益—风险前沿

最终测试改用从未参与选模的种子 20260821，每种天气生成 20000 条新序列。设 I_j 为第 j 条序列的失败指示， Y_j 为失败记 0 后的终值，则报告

$$\hat{p} = \frac{1}{N} \sum_{j=1}^N I_j, \quad \hat{\mu} = \frac{1}{N} \sum_{j=1}^N Y_j. \quad (34)$$

失败率使用 Wilson 区间，均值使用样本标准误差区间；完整公式和图形检验见附录。

表 4 第四关入选策略的独立样本外结果

天气模型	失败率	95% Wilson 区间	总体均值/元	总体 5% 分位/元
低沙暴	0	[0, 0.0192%]	10358.8	8880.0
基准	0.055%	[0.0307%, 0.0985%]	10162.9	8805.0
不利	0.385%	[0.3082%, 0.4809%]	9635.3	8627.5

基准天气下，入选 B5 比原 B6 策略总体均值高 418.3 元；不利天气下高 391.9 元，并保持失败率上置信界 0.481% 低于 0.5% 预算。B4 虽有更高成功者均值，但基准与不利失败率分别升至 0.145% 和 1.005%，不符合不利分布预算。

4.5 保持边际概率的 Markov 持续性扩展

独立同分布假设固定了每天的边际频率，却不能表示连续高温或连续沙暴。为把“天气聚集”与“天气占比”分离，令 $p = (p_s, p_h, p_b)$ 为前述边际分布，引入持续性参数 $\rho \in [0, 1)$ ，定义一阶 Markov 转移矩阵

$$P_\rho(z, z') = \rho \mathbf{1}_{\{z=z'\}} + (1 - \rho)p_{z'}. \quad (35)$$

当 $\rho = 0$ 时退化为 IID； ρ 增大时，次日以更高概率保持当天状态。该构造并未暗中改变沙暴边际概率，因为

$$(p^\top P_\rho)_{z'} = \rho p_{z'} + (1 - \rho)p_{z'} \sum_z p_z = p_{z'}, \quad (36)$$

故 p 仍是 P_ρ 的平稳分布。相邻两日天气相同的概率为

$$\Pr(Z_{t+1} = Z_t) = \rho + (1 - \rho) \sum_{z \in \mathcal{Z}} p_z^2, \quad (37)$$

从而 ρ 只控制序列聚集性，避免把相关性压力测试误写成“提高沙暴概率”。

题面没有转移矩阵，本文不估计一个虚假的单点 ρ ，而使用有限分布模糊集

$$\mathcal{P} = \{\mathbb{P}(p, \rho) : p \in \{p^{\text{base}}, p^{\text{adv}}\}, \rho \in \{0, 0.15, 0.35\}\}. \quad (38)$$

$\rho = 0.70$ 只作为集合外破坏性检验，不参与选模。对候选 θ ，分布鲁棒选择写为

$$\theta^{\text{DR}} = \arg \max_{\theta \in \Theta} \min_{\mathbb{P} \in \mathcal{P}} \hat{\mu}_{\mathbb{P}}(\theta) \quad \text{s.t.} \quad \hat{p}_{\mathbb{P}}(\theta) \leq \alpha(p), \quad \forall \mathbb{P} \in \mathcal{P}, \quad (39)$$

其中基准和不利边际的 α 分别为 0.2% 和 0.5%。每个模糊集点使用 500 条配对训练序列；在相同 567 候选上重做选模后，共有 8 个候选跨模糊集训练可行，最坏总体均值较高者为水 240 箱、食物 240 箱、 $B = 9$ 。随后冻结参数，在每个分布点用 30000 条独立序列验收。代表性端点见表 5。

表 5 第四关 B9 分布鲁棒策略的独立样本外结果

边际天气	ρ	失败率	95% Wilson 上界	总体均值/元
基准	0	0	0.0128%	8333.5
基准	0.35	0.0433%	0.0741%	8431.5
不利	0	0	0.0128%	7964.2
不利	0.35	0.1967%	0.2536%	8033.4

IID-B5 策略保留为“边际频率可信且无持续性”的效率基准，B9 策略则作为信息不足时的最终稳健推荐。两者差异不是算法冲突，而是保护范围扩大后的安全成本。值得注意的是，B9 在高持续性下的均值略高于 $\rho = 0$ ，原因是阈值策略会在成段晴朗期连续挖矿；风险控制仍由失败率上界而非均值单调性决定。

4.6 固定装载精确基准与风险对偶区间

为量化 IID 效率策略 B5 并非全局动态规划的代价，固定其实际运行条件：初始装载为 $(w_0, f_0) = (240, 240)$ ，已达 1200 千克负重上限，故不允许村庄补给；但保留完整 5×5 地图、30 天期限以及移动、停留、挖矿三类行动。此时天气揭示前状态为 $s = (t, l, w, f)$ ，终点价值为

$$G(s) = M_0 - 5w_0 - 10f_0 + R(s) + \frac{5}{2}w + 5f, \quad (40)$$

其中 $R(s)$ 为累计挖矿收益；未到终点、资源耗尽或超过期限均记为 0。于是固定装载子问题的风险约束最优值为

$$V_\alpha^* = \max_{\pi} \mathbb{E}[J_\pi] \quad \text{s.t.} \quad \Pr_\pi(F) \leq \alpha, \quad \alpha = 0.2\%. \quad (41)$$

这里 F 为失败事件。直接枚举连续概率预算会破坏普通 Bellman 递推，故用拉格朗日松弛给出严格上界：对任意 $\lambda \geq 0$,

$$V_\alpha^* \leq \max_{\pi} \left\{ \mathbb{E}[J_\pi] - \lambda \Pr_\pi(F) \right\} + \lambda\alpha =: U(\lambda). \quad (42)$$

右侧仍是标准天气先揭示 MDP，能精确遍历 798565 个状态。取 $\lambda = 79800$ 元，得到 $U(\lambda) = 11239.66$ 元；对应拉格朗日最优策略失败率为 0.1309%。另一方面，B5 的精确递推值为 10161.64 元、失败率为 $0.0307\% < 0.2\%$ ，因而它是式 (41) 的可行下界。综上

$$10161.64 \leq V_{0.2\%}^* \leq 11239.66, \quad \frac{11239.66 - 10161.64}{11239.66} = 9.59\%. \quad (43)$$

该区间衡量的是 B5 在固定装载、无补给、基准 IID 子问题中的风险约束动态决策缺口；它既不是整个第四关的全局最优区间，也不适用于 B9 的 Markov 分布鲁棒推荐。

问题二结论

第三关选择 54/54 箱三日直达，属于全支撑精确结论[E2]。第四关在 IID 假设下以 240/240 箱、B5 为效率方案；固定装载子问题中其风险约束最优性区间宽度为 9.59%。把连续天气纳入 $\mathcal{P}$ 后，最终推荐 240/240 箱、B9 阈值策略，属于明确模糊集和风险预算下的分布鲁棒样本外推荐[E4]。

5 问题三：多人拥挤博弈与在线反馈

5.1 多人规则的精确恢复

原题中的多人系数以旧版 Word 的 WMF 公式保存，普通文本抽取会遗漏。通过原始页面渲染核实：同一天同一有向边上有 $k \geq 2$ 人时，每人移动消耗系数由 2 变为 $2k$ ；同矿 k 人的消耗系数仍为 3，每人收入为 B/k ；同村至少两人购买时，单价由基准价 2 倍升为 4 倍。第五关 $n = 2$ ，第六关 $n = 3$ 。

对玩家 i ，给定联合行动 $\mathbf{a}_t$ ，移动消耗倍数为

$$\lambda_t^i = \begin{cases} 2k_{t,e}, & a_t^i = \text{沿有向边 } e \text{ 移动}, k_{t,e} \geq 2, \\ 2, & a_t^i = \text{移动且 } k_{t,e} = 1, \\ 3, & a_t^i = \text{挖矿}, \\ 1, & a_t^i = \text{停留}. \end{cases} \quad (44)$$

记 $k_{t,m}$ 为同矿挖矿人数， $k_{t,v}^B$ 为同村同日购买人数。玩家 i 的资源和现金转移可统一写成

$$r_t^i = r_{t-1}^i + b_t^{r,i,-} + b_t^{r,i,+} - \lambda_t^i c_{z_t}^r, \quad r \in \{w, f\}, \quad (45)$$

$$m_t^i = m_{t-1}^i + \frac{B}{k_{t,m}} I_{t,i}^M - \rho(k_{t,v}^B) \sum_r p_r b_t^{r,i}, \quad (46)$$

$$\rho(k) = \begin{cases} 4, & k \geq 2, \\ 2, & k = 1. \end{cases} \quad (47)$$

其中无挖矿时令 $I_{t,i}^M = 0$ 。联合状态 $\mathbf{S}_t = (S_t^1, \dots, S_t^n)$ 不再能按玩家独立转移，因为 λ_t^i 、矿山收益和采购价格都由联合行动决定。令 Φ 为“先分组、再结算”的联合映射，则

$$\mathbf{S}_t = \Phi(\mathbf{S}_{t-1}, z_t, \mathbf{a}_t, \mathbf{b}_t), \quad (48)$$

这也是多人模拟器逐日执行的数学定义。

这使玩家收益 $J_i(\pi_i, \pi_{-i})$ 显式依赖其他玩家策略。Nash 均衡要求

$$J_i(\pi_i^*, \pi_{-i}^*) \geq J_i(\pi_i, \pi_{-i}^*), \quad \forall i, \forall \pi_i, \quad (49)$$

与非合作博弈的经典定义一致 [5]。

5.2 第五关：完整最佳响应动态规划

天气和对手计划固定时，自身最优响应仍可用动态规划求解。状态为

$$(t, l_t, W_t, F_t),$$

其中 W_t, F_t 是截至第 t 日累计需要购买的资源。给定对手第 t 日的位置—行动 (l_t^{-i}, a_t^{-i}) ，自身行动 a 产生确定增量

$$(\Delta W_t, \Delta F_t, \Delta R_t) = g_t(l, a; l_t^{-i}, a_t^{-i}, z_t), \quad (50)$$

其中同向边把移动倍数由 2 改为 4，同矿把收入由 200 元改为 100 元。定义 $D_t(l, W, F)$ 为到达该标签时可得的最大累计矿山收入，则标签递推为

$$D_t(l', W', F') = \max_{a \in \mathcal{A}_t(l)} \{D_{t-1}(l, W, F) + \Delta R_t(a)\}, \quad (51)$$

其中 $l' = L(l, a)$ 、 $W' = W + \Delta W_t$ 、 $F' = F + \Delta F_t$ 。只保留满足

$$3W' + 2F' \leq C, \quad p_w W' + p_f F' \leq M_0, \quad d(l', v_T) \leq T - t \quad (52)$$

的标签。相同 (l', W', F') 只保留 D_t 最大的路径，因为它对未来可行域无影响而终值更高。终点标签的价值为

$$J_i = M_0 - p_w W_\tau - p_f F_\tau + D_\tau(v_T, W_\tau, F_\tau). \quad (53)$$

由有限日期、有限节点和整数容量可知标签集有限，故式 (51) 枚举了固定对手下的完整策略空间，而不是局部换路启发式。

从单人最优计划出发交替计算最佳响应，在 30 轮上限内收敛；随后固定对手，对双方各再做一次完整最佳响应。定义偏离增益

$$\Delta_i = \max_{\pi_i} J_i(\pi_i, \pi_{-i}^*) - J_i(\pi_i^*, \pi_{-i}^*). \quad (54)$$

只有当 $\Delta_1 = \Delta_2 = 0$ 时才接受纯策略 Nash 均衡。结果见表 6。

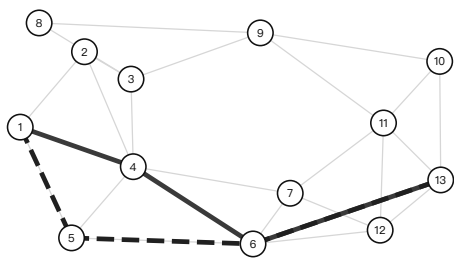
表 6 第五关非对称纯策略 Nash 均衡

玩家	初始水/食物	行动序列	到达日	终值/元	最大偏离增益/元
1	27/33	1 → 4, 停留, 4 → 6 → 13	4	9535	0
2	30/34	1 → 5 → 6 → 13	3	9510	0

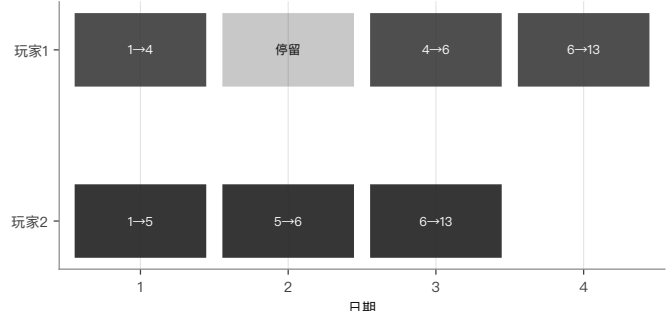
第 2 日是高温，玩家 1 在 4 区停留，把两个移动日推迟到第 3、4 日晴朗；玩家 2 从 5 区走另一条首段路线。两人的同日有向边没有重叠。低基础收益使绕矿不具吸引力，因此均衡核心是“天气错峰 + 空间分流”，而不是抢矿。

第五关非对称纯策略 Nash 均衡

空间路径：同日有向边不重合



时序均衡：9535元 vs 9510元



完整最佳响应复核：两名玩家最大单边偏离增益均为0元

图 6 第五关均衡的空间路径、行动时序与偏离复核

5.3 第六关：三人联合反馈模拟

第六关每天先揭示天气，再由三名玩家根据上一日公开的联合状态选择行动。联合反馈策略是映射

$$\pi_t : (S_0, z_1, a_1, \dots, S_{t-1}, z_t) \mapsto a_t, \quad (55)$$

其后由式 (48) 一次性结算。模拟器严格按行动前村庄购买分组、同向边分组、同矿分组、资源扣除、收益分摊、行动后购买、失败和终点结算的顺序执行。为分离不同外部性，比较四种策略：

1. **镜像挖矿**：三人复制单人矿山阈值策略；
2. **分路直达**：两人分别走上侧和左侧 8 日最短路，第三人错峰 1 个非沙暴日后跟随；
3. **拥挤感知非合作**：先分路到矿山，在场且资源安全者都加入挖矿；
4. **合作轮换**：先分路到矿山，每天最多安排一人挖矿，作为可协调对照。

先在 IID 假设下建立效率基准。分路直达的初始水食从 180 到 220 箱、步长 5 搜索，共 9 个候选。设装载参数 q 表示每人初始水、食物均为 q 箱，训练选择为

$$q^* = \arg \max_{q \in \{180, 185, \dots, 220\}} \hat{\mu}_{\text{adv}}(q) \quad \text{s.t.} \quad \hat{p}_{\text{base}}(q) \leq 0.002, \hat{p}_{\text{adv}}(q) \leq 0.005. \quad (56)$$

训练阶段基准和不利分布各 3000 局；入选 $q^* = 185$ 。测试阶段换用种子 20260921，每种分布 20000 局，即 60000 个玩家观测。因为同一局三人的失败相关，玩家级区间用于描述个体风险，同时另行计算局级区间，避免把 60000 个观测误当作完全独立的局。

第六关：分路直达是唯一跨分布满足风险预算的策略

每点20000条天气、60000个玩家观测；失败终值记为0；横轴为对称对数尺度

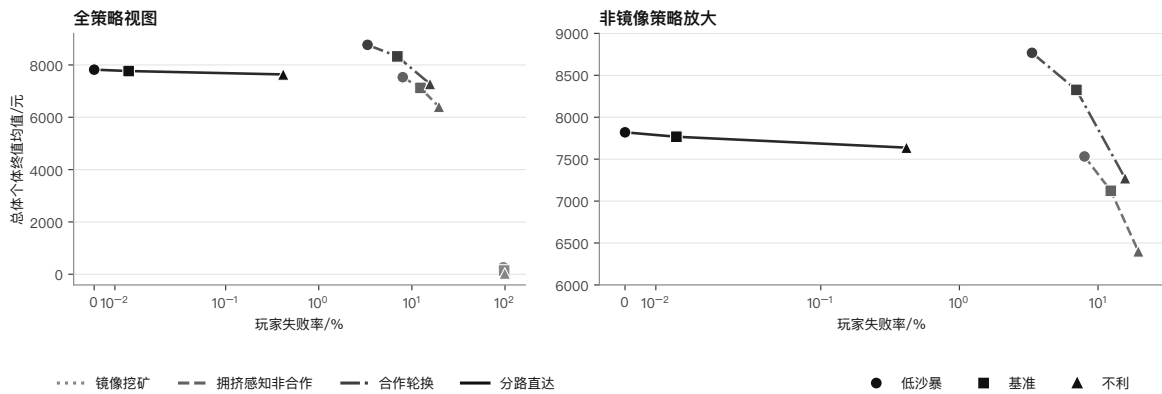


图 7 第六关四类反馈策略跨天气分布的失败风险与总体终值

表 7 第六关分路直达策略的独立样本外结果

天气模型	玩家失败率	95% Wilson 区间	总体均值/元	5% 分位/元
低沙暴	0	[0, 0.0064%]	7820.3	7612.5
基准	0.0167%	[0.0091%, 0.0307%]	7767.6	7542.5
不利	0.415%	[0.3666%, 0.4697%]	7637.4	7395.0

基准分布下，镜像挖矿、拥挤感知非合作、合作轮换的玩家失败率分别为 97.865%、12.362% 和 6.978%；总体均值分别为 142.6、7122.8 和 8326.6 元。合作轮换均值高于直达，却远超 0.2% 风险预算；故 IID 层面的风险受控基准是分路直达，下一节再检验其面对天气持续性的稳定性。

5.4 角色条件装载与分布鲁棒策略

185/185 箱是 IID 模型下的效率点，但连续坏天气会使它越过风险预算。沿用式 (38) 的模糊集，将最短路角色按出发前自愿错峰天数 d 分类。对 $d \in \{0, 1, 2\}$ ，在水、食物 180–240 箱、步长 5 的二维网格 $\mathcal{L}$ 上校准

$$(w_d^*, f_d^*) = \arg \max_{(w, f) \in \mathcal{L}} \min_{\mathbb{P} \in \mathcal{P}} \hat{\mathbb{E}}_{\mathbb{P}}[J(w, f, d)] \quad \text{s.t.} \quad U_W(X_{\mathbb{P}}; N) \leq \alpha(p), \quad \forall \mathbb{P} \in \mathcal{P}, \quad (57)$$

其中 U_W 是式 (65) 和 (66) 给出的失败率 Wilson 上界，而非失败率点估计。没有边拥挤时，八个移动日加 d 个非沙暴停留日的资源需求可直接写成

$$R_w(z_{1:\tau}; d) = \sum_{t=1}^{\tau} \lambda_t(d) c_{z_t}^w, \quad R_f(z_{1:\tau}; d) = \sum_{t=1}^{\tau} \lambda_t(d) c_{z_t}^f, \quad (58)$$

$$\lambda_t(d) = \begin{cases} 1, & z_t = \text{沙暴或当日用于自愿错峰}, \\ 2, & \text{当日沿最短路移动}, \end{cases} \quad (59)$$

到达所需的第八个非沙暴移动日为 τ 。若 $R_w > w$ 或 $R_f > f$ 则失败记 0；否则终值为

$$J = 10000 - \frac{1}{2}p_w w - \frac{1}{2}p_f f - \frac{1}{2}p_w R_w - \frac{1}{2}p_f R_f, \quad (60)$$

因为未消耗库存按半价回收。该式也解释了为什么在通过风险约束后，继续加装会单调降低期望终值。

每个模糊集点使用 30000 条校准序列。 $d = 0$ 的最优可行装载为 215/215 箱， $d = 1$ 为 225/230 箱； $d = 2$ 在 1200 千克容量内没有同时通过两档风险预算的装载。因此最终剖面为两名玩家立即从不同方向出发、各装 215/215 箱，第三名玩家装 225/230 箱并错峰一个非沙暴日。公开随机置换三个角色后，每个人承担相同的事前混合风险，但角色条件装载还额外保证每个固定角色单独过线。

表 8 第六关分布鲁棒剖面的独立样本外结果

边际天气	ρ	玩家失败率	固定角色最大 Wilson 上界	总体均值/元
基准	0	0	0.0128%	7509.6
基准	0.35	0.0600%	0.1030%	7499.4
不利	0	0.0056%	0.0390%	7410.0
不利	0.35	0.3400%	0.4344%	7383.3

表中每档 30000 局。“玩家失败率”按三角色合并统计；验收列则分别对三个固定角色各用 30000 个独立天气样本计算 Wilson 上界，再取最大值，避免把同局三人的相关结果误当作 90000 个独立 Bernoulli 观测。模糊集内最坏固定角色上界 0.4344% 仍低于 0.5% 预算。

角色条件二维装载校准：风险上置信界决定可行域

黑点同时通过基准0.2%与不利0.5%预算；黑星最大化模糊集内最坏总体均值

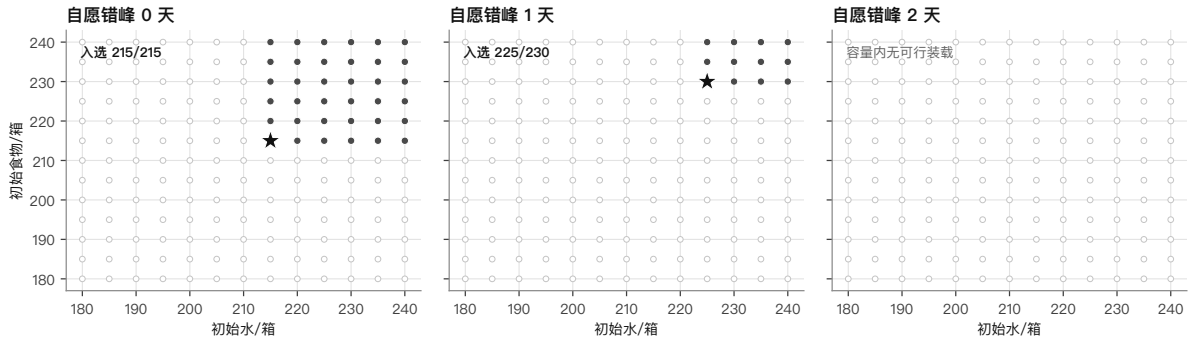


图 8 第六关错峰角色的二维装载可行域与分布鲁棒入选点

5.5 受约束最佳响应预言机与可利用度

5 乘 5 网格从 1 到 25 的最短路需要 4 次向右和 4 次向下，因此共有

$$|\mathcal{R}| = \binom{8}{4} = 70 \quad (61)$$

条。构造偏离类 Π_i^C ：玩家 i 可在 70 条最短路中任意换路，并选择 $d = 0$ 或 1；装载使用式 (57) 为该 d 校准的风险最优装载。 $d = 2$ 因不存在风险可行装载而剔除。于是每名玩家有 $70 \times 2 = 140$ 个可审计偏离，而对手保持推荐剖面。

令分布鲁棒收益 $\underline{J}_i(\pi) = \min_{\mathbb{P} \in \mathcal{P}} \mathbb{E}_{\mathbb{P}}[J_i(\pi)]$ ，定义受约束可利用度

$$\epsilon_i^C = \max_{\pi'_i \in \Pi_i^C: U_W \leq \alpha} \underline{J}_i(\pi'_i, \pi_{-i}^*) - \underline{J}_i(\pi_i^*, \pi_{-i}^*). \quad (62)$$

预言机先在经验最不利角点“不利边际、 $\rho = 0.35$ ”用 1500 条配对序列完整筛选，再在 $\mathcal{P}$ 每个点各用 30000 条新序列复核入选偏离。三个角色的最佳响应都只是换到另一条不发生同向边重合的等价最短路，样本外收益与原角色逐样本相同，故 $\epsilon_i^C = 0$ 。这把原先

的“比较四种策略”推进到显式偏离检验；但 Π_i^C 不含绕矿、村庄补给及任意历史依赖策略，因此结论是类约束 ϵ -均衡，而非整个随机动态博弈的 Markov 完美均衡。

第六关受约束最佳响应预言机：140个偏离/角色

最坏集合点：不利边际、 $\rho = 0.35$ ；搜索70条最短路×2种可行错峰装载

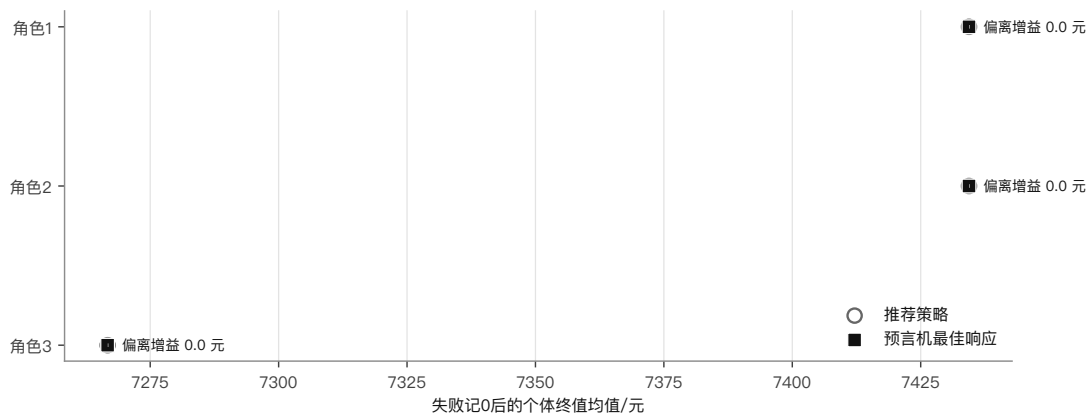


图 9 第六关推荐剖面与受约束最佳响应的样本外收益比较

5.6 外部性分解与角色公平

图 10给出基准天气的机制量。镜像挖矿每局产生 19.44 个共享边玩家日，三人往往尚未到矿山就因 6 倍移动消耗失败。拥挤感知非合作把共享边降至 4.02，却产生 8.74 个共享矿山玩家日；合作轮换消除共享矿山，但等待时间抬高尾部失败。分路直达两类共享玩家日均为 0。

第六关失效机制：镜像策略把单人路径放大为共享边灾难

基准天气，20000局；分路直达的共享边与共享矿山玩家日均为0

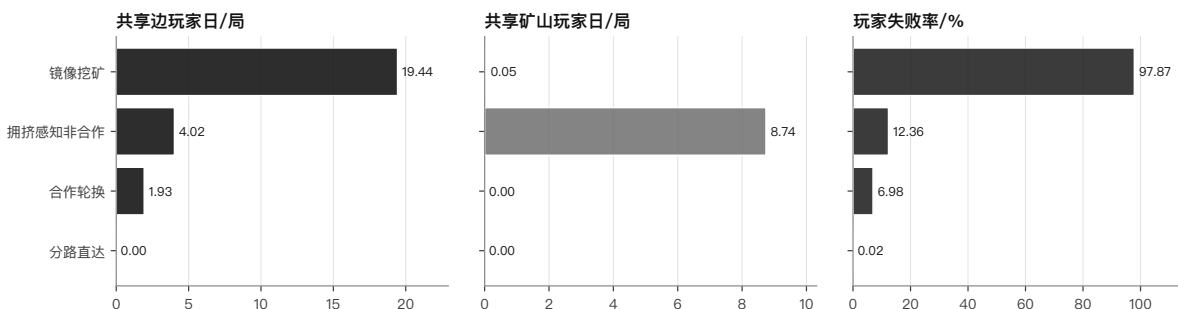


图 10 第六关多人策略的共享边、共享矿山与失败率分解

分布鲁棒装载下，第三名玩家因错峰与更高安全库存，在基准 IID 分布平均比前两名少 155.2 元；不利边际、 $\rho = 0.35$ 时角色差为 165.9 元。设三个路线角色的期望终值为 μ_1, μ_2, μ_3 ，第 0 天生成与身份无关的均匀随机置换 σ ，把玩家 i 分配到角色 $\sigma(i)$ 。则对

任意玩家 i ,

$$\mathbb{E}[J_i] = \sum_{r=1}^3 \Pr\{\sigma(i) = r\} \mu_r = \frac{1}{3}(\mu_1 + \mu_2 + \mu_3) = \bar{\mu}. \quad (63)$$

所以随机置换不改变三条路线的联合占用和系统总收益，却使每名玩家获得相同的事前期望。固定角色的事后收益仍可能不同，这里主张的是事前公平而非逐局完全相等。

问题三结论

第五关的 9535/9510 元组合通过完整最佳响应检验，属于精确纯策略均衡[E3]。第六关在 IID 下以 185/185 箱为效率基准；最终分布鲁棒推荐采用 215/215、215/215 与 225/230 箱的异路错峰剖面，并在每名玩家 140 个路线—错峰偏离类中得到零可利用度[E4]。该结论不外推为完整 Markov 完美均衡。

6 模型检验、敏感性与适用边界

6.1 数值一致性和独立回放

计算链分为“优化/决策器”和“规则模拟器”两个入口。问题一由 MILP 产生行动变量，再由不参与优化的逐日模拟器检查；第三关 Bellman 值与 1024 条全序列加权回放一致；第五关均衡计划由独立的路径评价器重算资源和收益；第四、六关所有 Monte Carlo 样本均逐日执行同一规则内核。当前 27 项自动测试覆盖地图连通与对称、沙暴禁行、非矿山挖矿拒绝、起点超重拒绝、村庄行动前购买、已知天气参考流水、在线天气复现、Markov 平稳性、70 条最短路枚举和多人同边消耗。

定义约束残差 $\varepsilon_g = \max\{0, g(x)\}$ (对 $g(x) \leq 0$) 和 $\varepsilon_h = |h(x)|$ (对 $h(x) = 0$)。整数计划回放后，资源、现金、负重和位置的最大残差均为 0；第三关独立枚举均值与 Bellman 值之差小于 10^{-6} ；第五关两名玩家的偏离增益均为 0。验收不是只比较最终答案，而是逐层比较状态轨迹和约束残差。

表 9 主要模型的数值验收准则与结果

对象	验收量	阈值	实测结果
第一、二关 MILP	求解状态；逐日规则残差	Optimal; 0	Optimal; 0
第三关 MDP	Bellman 值与 1024 序列加权均值差	$< 10^{-6}$	$< 10^{-6}$
第五关均衡	$\max_i \Delta_i$	0	0 元
第四、六关	选模样本与测试样本是否重合	0 条	0 条
全模型	自动化规则测试	全部通过	27/27

6.2 训练—测试隔离与幸存者偏差

第四关 IID 训练种子为 20260721，最终测试种子从 20260821 开始；分布鲁棒实验统一从 20261021 派生互不重叠的天气验证、装载校准、预言机筛选和最终测试随机流。

候选选择只读取训练结果，测试输出不反向修改风险预算。设成功样本终值为 J_j ，失败指示为 F_j ，本文的总体估计量为

$$\hat{\mu} = \frac{1}{N} \sum_{j=1}^N (1 - F_j) J_j, \quad (64)$$

而不是只在 $F_j = 0$ 的子样本上取均值。故高收益、高失败的策略不会因剔除失败局而显得虚假优越。失败率区间采用 Wilson 得分区间 [7]。

6.3 失败率与均值区间推导

设 X 次失败来自 N 个 Bernoulli 观测， $\hat{p} = X/N$ ， $z = z_{0.975} = 1.959964$ 。直接使用 Wald 区间 $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/N}$ 会在 $X = 0$ 时退化为零宽区间，因此采用 Wilson 得分区间。其中心和半宽为

$$c_W = \frac{\hat{p} + z^2/(2N)}{1 + z^2/N}, \quad (65)$$

$$h_W = \frac{z}{1 + z^2/N} \sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}, \quad (66)$$

故区间为 $[\max(0, c_W - h_W), \min(1, c_W + h_W)]$ 。例如第四关低沙暴样本 $X = 0, N = 20000$ ，仍得到非零上界 0.0192%；这比宣称“风险为 0”更审慎。

对失败记 0 后的终值 $Y_1, \dots, Y_N$ ，记样本均值 $\bar{Y}$ 、样本标准差 s_Y ，大样本 95% 区间为

$$\bar{Y} \pm z \frac{s_Y}{\sqrt{N}}. \quad (67)$$

第六关同时输出玩家级与局级失败区间：前者回答“随机玩家失败概率”，后者回答“至少一名玩家失败的局概率”。两种分母和问题不同，不能混用。

6.4 第四关策略缺口的精确对偶验收

除 Monte Carlo 样本外检验外，第四关还在 240/240 满负重、无补给的共同子问题内以精确 MDP 复核 B5。B5 的递推值和失败率分别为 10161.64 元、0.0307%，与 20000 条独立样本的 10162.9 元、0.055% 一致。对风险预算 $\alpha = 0.2\%$ ，由式 (42) 构造的 $\lambda = 79800$ 对偶上界为 11239.66 元，故 B5 并非被直接宣称为全局动态最优，而是在相同装载、行动空间和风险预算下有宽度 9.59% 的可证区间。该检验同时排除了“无约束期望最大”策略：它虽可获得更高收益，却会突破预设失败预算。

6.5 参数敏感性与安全裕度

附录图 13 先复核 IID 效率方案：第四、六关在 IID 不利天气下的上置信界均低于 0.5%。图 14 显示，六关统一装载从 180 增至 220 箱时，失败率下降而终值近似线性下降，解释了 IID 下 185/185 的局部选择。

更严格的持续性压力检验见图 11。当边际概率不变而 ρ 从 0 升至 0.35 时，IID 效率方案的连续坏天气暴露明显放大：第四关 B5 和第六关 185/185 均越过风险预算；分布鲁棒方案在 $\rho \leq 0.35$ 的全部六个模糊集点仍通过 Wilson 上界。 $\rho = 0.70$ 下两者均可能越界，这一集合外结果不是被隐藏的坏点，而是明确给出模型保护范围的失效边界。

连续天气压力检验：保持边际概率不变，只提高时间聚集性

灰底为分布模糊集 $\rho \leq 0.35$ ； $\rho = 0.70$ 为集合外破坏性测试；0 值置于 0.005% 绘图下限

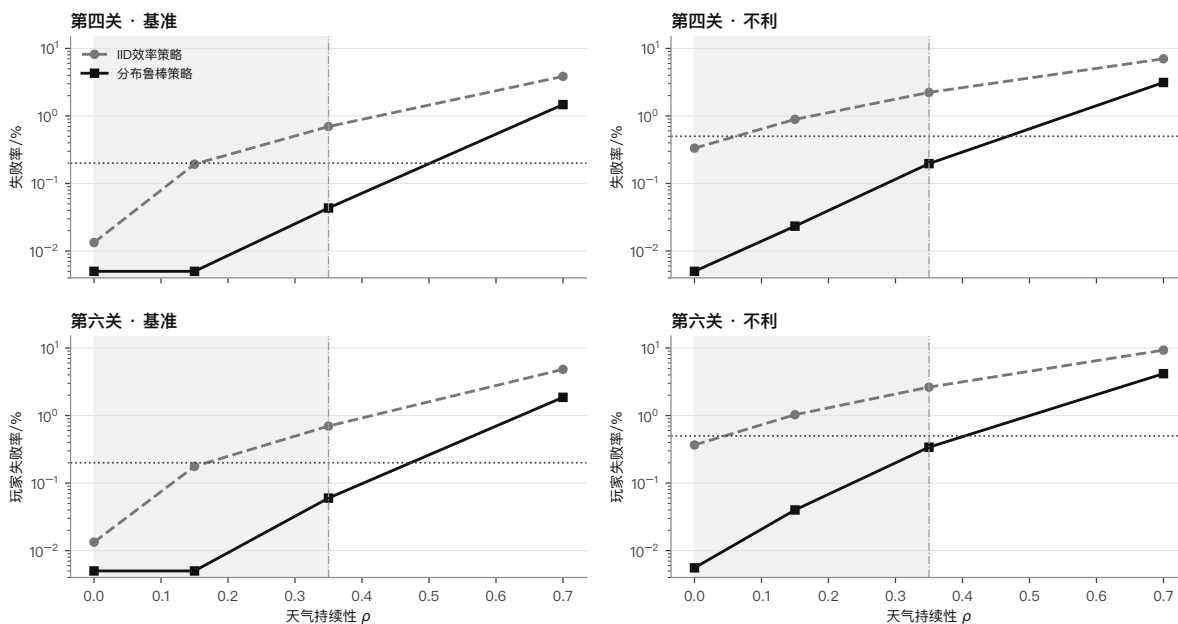


图 11 保持天气边际分布不变时，持续性对第四、六关失败风险的影响

6.6 消融：为什么不能复制单人策略

第六关镜像挖矿是对“忽略交互项”的直接消融。单人第四关 B5 在基准天气总体均值 10162.9 元、失败率 0.055%；三人原样复制后，基准失败率升至 97.865%，总体均值降到 142.6 元。变化并非天气或地图造成，而是同边移动从 2 倍升为 6 倍。进一步从镜像改为分路但保留矿山竞争，失败率降至 12.362%；再彻底放弃矿山并分路直达，失败率降至 0.0167%。这一逐层消融把外部性来源从相关性转化为可解释机制。

6.7 六关证据矩阵

六关结论与证据等级总览		
关卡	核心结果	证据等级
第一关	10470元	E1 最优证书
第二关	12730元	E1 最优证书
第三关	54/54直达	E2 全枚举
第四关	B9分布鲁棒	E4 样本外
第五关	9535/9510元	E3 偏离复核
第六关	角色装载： $\epsilon=0$	E4 样本外

E1/E2为精确结论；E3为完整最佳响应检验；E4为预注册风险预算下的独立样本外推荐

图 12 六关核心结果与结论证据等级

6.8 模型优点

- 信息结构一致。**三类模型都明确天气在行动前或行动后何时可见，杜绝未来信息泄漏；随机规划中的逐期信息适应原则与多阶段随机优化框架一致 [4]。
- 结论强度可审计。**全局最优、全枚举、最佳响应和样本外推荐使用不同证据标签，评审可快速识别哪些结论依赖概率假设。
- 失败被纳入效用。**选模同时控制均值和失败风险，避免只比较到达者。
- 一般性强。**地图、天气、资源与多人系数通过场景对象注入；更换附件无需重写规则内核。
- 机制可解释。**同边玩家日、同矿玩家日、库存和现金流水把结果落回题面规则，而非黑箱给数。
- 分布偏移可诊断。** Markov 构造保持天气边际不变，只改变聚集性；因此可以把概率偏差与时序相关的影响分开归因，并用集合外压力点展示安全边界。

6.9 局限与可证伪条件

分布边界：题面只说“较少沙暴”，没有给出概率或相关性。本文用两个边际分布与 $\rho \leq 0.35$ 构成可审计保护集，而非声称它是真实分布。若官方转移矩阵落在集合外，应扩大 $\mathcal{P}$ 并重新校准； $\rho = 0.70$ 压力点已经说明当前容量下无法保证同一风险预算。

均衡边界：第五关完成了有限策略空间的完整最佳响应，结论可称 Nash 均衡；第六关把偏离证据扩展到每名玩家 140 个最短路—错峰策略并得到类约束零可利用度，但未覆盖绕矿、补给与任意历史依赖偏离，不能称 Markov 完美均衡。

行为边界：公开随机角色分配实现事前公平，但题面没有强制协调机制。若玩家拒绝角色约定，应使用非合作动态博弈重新求解。

数据边界：地图由附件图形人工重建并经拓扑测试；若原图邻接判定有误，最短路与全部结果都会改变。节点邻接表因此作为可审计数据随代码交付。

最可能推翻推荐的三类新信息是：官方给出天气转移矩阵；第六关允许有约束力的合作和转移支付；附件地图的邻接边被官方修正。任一变化发生时，应保留规则模拟器，仅替换天气模型、博弈机制或邻接表后重跑。

7 结论

本文以统一图网络和资源—现金守恒为底座，按照信息集从全知、当日知到多人反馈依次建立 MILP、MDP 与动态博弈，完成三个问题和六关的建模、求解与验证。

1. **确定天气：**第一、二关全局最优终值分别为 10470 元和 12730 元。最优性证书之外，逐日回放证明路线、补给、挖矿、现金和负重全部合法，并纠正第二关公开流水的一处水量错误。
2. **仅知当天气象：**第三关稳健最优策略为 54/54 箱三日直达，在三档高温概率和各 1024 条支撑天气中零失败。第四关 IID 效率点为 240/240 箱、B5；保持边际不变并允许 $\rho \leq 0.35$ 后，分布鲁棒解为 240/240 箱、B9。在不利边际、 $\rho = 0.35$ 下失败率 0.1967%，Wilson 上界 0.2536%。
3. **多人交互：**第五关的非对称均衡终值为 9535 和 9510 元，双方完整单边偏离增益均为 0。第六关证明机械复制单人挖矿策略会因共享边外部性几乎必败；分布鲁棒推荐为 215/215、215/215 和 225/230 箱的异路错峰剖面，在不利边际、 $\rho = 0.35$ 下玩家失败率 0.3400%、总体均值 7383.3 元；每名玩家 140 个风险校准偏离的最大可利用度为 0 元。

最重要的管理含义是：天气不确定时不仅要估计天气出现频率，还要防范连续坏天气；多人环境下应先消除同向边重合，再按角色暴露校准装载，不能被单人矿山高收益诱导。方法上的核心则是把模型输出、规则合法性、分布偏移、统计风险和均衡稳定性分开验证。由此，论文的六关结论既可复现，也清楚说明了各自能被信任到什么程度。

最终推荐

第一关执行 10470 元最优计划；第二关执行 12730 元最优计划；第三关 54/54 箱直达；第四关采用 240/240 箱、B9 分布鲁棒阈值策略（若确认天气 IID 可改用 B5 效率方案）；第五关采用异路错峰纯策略均衡；第六关采用 215/215、215/215 与 225/230 箱的异路错峰剖面，并在出发前公开随机分配三个角色。

参考文献

- [1] Bellman R. *Dynamic Programming*[M]. Princeton: Princeton University Press, 1957.
- [2] Bertsimas D, Tsitsiklis J N. *Introduction to Linear Optimization*[M]. Belmont: Athena Scientific, 1997.
- [3] Puterman M L. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*[M]. New York: Wiley, 1994. DOI: 10.1002/9780470316887.
- [4] Shapiro A, Dentcheva D, Ruszczyński A. *Lectures on Stochastic Programming: Modeling and Theory*[M]. 2nd ed. Philadelphia: SIAM, 2014. DOI: 10.1137/1.9781611973433.
- [5] Nash J. Non-cooperative games[J]. *Annals of Mathematics*, 1951, 54(2): 286–295. DOI: 10.2307/1969529.
- [6] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. *Mathematical Programming Computation*, 2018, 10(1): 119–142. DOI: 10.1007/s12532-017-0130-5.
- [7] Wilson E B. Probable inference, the law of succession, and statistical inference[J]. *Journal of the American Statistical Association*, 1927, 22(158): 209–212. DOI: 10.1080/01621459.1927.10502953.
- [8] 全国大学生数学建模竞赛组委会. 2020 年高教社杯全国大学生数学建模竞赛 B 题：穿越沙漠 [Z]. 2020.

A 可复现算法与逐日流水

A.1 核心算法伪代码

算法 A：确定天气时空网络

1. 从场景对象读取邻接图、天气、特殊节点与资源参数；
2. 创建日期—位置二元流变量、行动变量、购买量、库存和现金变量；
3. 加入每日唯一行动、位置流、沙暴禁行、矿山、村庄、资源、现金和负重约束；
4. 最大化终点现金与半价回收，调用 HiGHS；
5. 把最优变量转换为逐日计划，交独立模拟器回放；任何规则不一致则拒绝输出。

算法 B：天气先揭示的有限域 MDP

1. 枚举所有满足现金与负重的整数初始采购；
2. 从截止日反向计算 $V_t(t, l, w, f)$ ，对当天每种支撑天气先观察再选行动；
3. 若任一正概率后继无法按时到达，则当前动作不可行；
4. 在第 0 日选择期望值最大的采购，并保存确定性策略表；
5. 对全部天气支撑逐条回放，复核 Bellman 值和失败数。

算法 C：第五关完整最佳响应

1. 固定对手逐日计划，构造其“日期—起点—行动”时间表；
2. 在状态 (t, l, W, F) 上枚举移动、停留、挖矿，并按同边/同矿规则计算自身增量；
3. 对相同状态只保留挖矿收入最高标签，用最短距离剪去无法到达状态；
4. 交替求两人的最佳响应直至计划不变；
5. 固定最终对手计划再完整求解一次，报告最大偏离增益。

算法 D：风险约束训练—测试流程

1. 预先固定天气模型、风险预算、参数网格、训练种子和评价指标；
2. 对所有候选使用同一批配对天气，计算含失败总体均值和失败率；
3. 先过滤风险不合格候选，再在可行集中最大化不利分布总体均值；
4. 冻结入选参数，切换未参与选模的测试种子和更大样本；
5. 同时输出点估计、Wilson 区间、均值区间和尾部分位数，不以测试结果回调参数。

算法 E：三人联合反馈结算

1. 读取共同天气和上日日末联合状态，三名玩家同步提交行动；
2. 按村庄分组结算行动前采购，按有向边和矿山分别建立拥挤组；
3. 由组人数确定移动倍数、矿山分成，逐人扣除资源并判定失败或到达；
4. 按村庄重新分组结算行动后采购，写入不可变逐日记录；

5. 截止后将失败记 0，对成功者结算现金和剩余物资半价回收。

算法 F：分布鲁棒天气—装载—偏离闭环

1. 由两档天气边际和 $\rho \in \{0, 0.15, 0.35\}$ 生成保持平稳边际的 Markov 模糊集；

2. 对第四关 567 候选最大化模糊集内最坏总体均值，并在全部集合点约束失败预算；

3. 对第六关错峰 0、1、2 日角色枚举二维装载，以固定角色 Wilson 上界筛选；

4. 枚举 70 条最短路与风险可行错峰装载，对三名玩家分别调用最佳响应预言机；

5. 换用独立随机流复核六个集合点，并用 $\rho = 0.70$ 给出保护集外失效边界。

A.2 第一、二关完整逐日流水

表10由output/problem1/known_weather_exact.json 自动生成；购买栏为“水/食物”。

表 10 第一、二关全局最优计划的逐日审计流水

关卡	日	天气	位置/行动	现金	水	食物	购买 (水/食)
1	0	—	1/初始	5780	178	333	0/0
1	1	高温	25/行走	5780	162	321	0/0
1	2	高温	26/行走	5780	146	309	0/0
1	3	晴朗	23/行走	5780	136	295	0/0
1	4	沙暴	23/停留	5780	126	285	0/0
1	5	晴朗	21/行走	5780	116	271	0/0
1	6	高温	9/行走	5780	100	259	0/0
1	7	沙暴	9/停留	5780	90	249	0/0
1	8	晴朗	15/行走	4150	243	235	163/0
1	9	高温	13/行走	4150	227	223	0/0
1	10	高温	12/行走	4150	211	211	0/0
1	11	沙暴	12/挖矿	5150	181	181	0/0
1	12	高温	12/挖矿	6150	157	163	0/0
1	13	晴朗	12/挖矿	7150	142	142	0/0
1	14	高温	12/挖矿	8150	118	124	0/0
1	15	高温	12/挖矿	9150	94	106	0/0
1	16	高温	12/挖矿	10150	70	88	0/0
1	17	沙暴	12/停留	10150	60	78	0/0
1	18	沙暴	12/停留	10150	50	68	0/0
1	19	高温	12/挖矿	11150	26	50	0/0
1	20	高温	13/行走	11150	10	38	0/0
1	21	晴朗	15/行走	10470	36	40	36/16

续下页

表 10（续）

关卡	日	天气	位置/行动	现金	水	食物	购买 (水/食)
1	22	晴朗	9/行走	10470	26	26	0/0
1	23	高温	21/行走	10470	10	14	0/0
1	24	晴朗	27/行走	10470	0	0	0/0
2	0	—	1/初始	5300	130	405	0/0
2	1	高温	9/行走	5300	114	393	0/0
2	2	高温	18/行走	5300	98	381	0/0
2	3	晴朗	26/行走	5300	88	367	0/0
2	4	沙暴	26/停留	5300	78	357	0/0
2	5	晴朗	35/行走	5300	68	343	0/0
2	6	高温	36/行走	5300	52	331	0/0
2	7	沙暴	36/停留	5300	42	321	0/0
2	8	晴朗	37/行走	5300	32	307	0/0
2	9	高温	38/行走	5300	16	295	0/0
2	10	高温	39/行走	5200	10	283	10/0
2	11	沙暴	39/停留	3460	174	273	174/0
2	12	高温	47/行走	3460	158	261	0/0
2	13	晴朗	55/行走	3460	148	247	0/0
2	14	高温	55/挖矿	4460	124	229	0/0
2	15	高温	55/挖矿	5460	100	211	0/0
2	16	高温	55/挖矿	6460	76	193	0/0
2	17	沙暴	55/挖矿	7460	46	163	0/0
2	18	沙暴	55/挖矿	8460	16	133	0/0
2	19	高温	62/行走	4730	201	207	201/86
2	20	高温	55/行走	4730	185	195	0/0
2	21	晴朗	55/挖矿	5730	170	174	0/0
2	22	晴朗	55/挖矿	6730	155	153	0/0
2	23	高温	55/挖矿	7730	131	135	0/0
2	24	晴朗	55/挖矿	8730	116	114	0/0
2	25	沙暴	55/挖矿	9730	86	84	0/0
2	26	高温	55/挖矿	10730	62	66	0/0
2	27	晴朗	55/挖矿	11730	47	45	0/0
2	28	晴朗	55/挖矿	12730	32	24	0/0
2	29	高温	56/行走	12730	16	12	0/0
2	30	高温	64/行走	12730	0	0	0/0

A.3 复现实验索引

表 11 关键命令、输入与输出

命令	主要输入	主要输出
<code>make test</code>	场景、模拟器、策略与博弈代码	27 项规则与回归测试
<code>make solve-known</code>	第一、二关地图和 30 天天气	<code>known_weather_exact.json</code>
<code>make analyze-problem2</code>	第三、四关场景	MDP 敏感性和手工策略基线
<code>make optimize-level4</code>	567 候选、训练/测试种子	第四关搜索和 20000 条样本外结果
<code>make analyze-problem3</code>	第五、六关多人场景	均衡、装载搜索、四策略比较
<code>make analyze-robustness</code>	Markov 模糊集、567 候选、70 条最短路	压力曲线、角色装载、可利用度
<code>make figures</code>	全部 JSON/CSV	15 张黑白 PDF/PNG 图
<code>make result</code>	精确流水、原 Result 模板	最终 <code>Result.xlsx</code>
<code>make deliver</code>	全部计算结果与论文源文件	最终 PDF 与 Excel 交付物

随机实验的训练、测试种子和样本数均写入对应 JSON；所有输出表使用 UTF-8 with BOM 以便 Excel 直接打开。最终论文引用的图由脚本白名单同步到 `paper/figures/generated/`，其余试验图只保留在 `output/candidate_figures/`，避免候选图混入交付。

B 统计检验、敏感性与公平性附图

B.1 样本外失败率区间

图 13 复核 IID 效率方案的失败率点估计、95% Wilson 区间和选模前固定的风险预算。第四关每档天气使用 20000 条序列；第六关每档天气使用 20000 局、60000 个玩家观测。两关在 IID 不利天气下的区间上界分别为 0.4809% 和 0.4697%，仍低于 0.5% 预算；该图用于解释旧效率点，最终分布鲁棒验收另见表 8 和图 11。

样本外风险点估计与上置信界同时通过预算

虚线为基准0.2%预算，点线为不利0.5%预算；低沙尘仅作敏感性参照

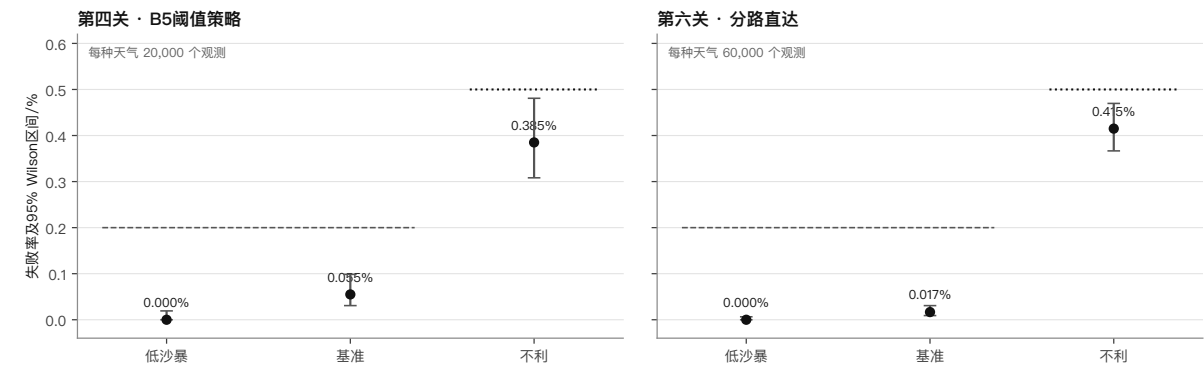


图 13 第四、六关入选策略的样本外失败率与 95% Wilson 区间

表 12 风险预算的区间上界复核

关卡	天气模型	点估计/%	95% 上界/%	预算/%
第四关	基准	0.0550	0.0985	0.2000
第四关	不利	0.3850	0.4809	0.5000
第六关	基准	0.0167	0.0307	0.2000
第六关	不利	0.4150	0.4697	0.5000

B.2 天气持续性构造的数值复核

对每个边际分布和 $\rho \in \{0, 0.15, 0.35, 0.70\}$ 生成 20000 条 30 日序列。程序同时输出沙尘边际占比、相邻天气相同比例、最长连续沙尘长度和连续至少 3 日沙尘的频率。两项核心验收为：各 ρ 下沙尘占比仍围绕给定 p_b 波动；经验相同天气率与式 (37) 的理论值一致。由此确认压力曲线的变化来自时间相关性，而不是天气边际被代码意外改写。

表 13 Markov 持续性实验的理论相同天气概率

边际模型	$\rho = 0$	$\rho = 0.15$	$\rho = 0.35$	$\rho = 0.70$
基准 (0.45, 0.45, 0.10)	0.4150	0.5028	0.6198	0.8245
不利 (0.35, 0.50, 0.15)	0.3950	0.4858	0.6068	0.8185

B.3 分布鲁棒装载与偏离搜索审计

图 8 中的每个点对应一个真实二维装载组合，而非插值曲面。立即出发角色有 36 个可行组合，入选 215/215；错峰 1 日角色有 12 个，入选 225/230；错峰 2 日在容量内无可行点。这一结果揭示统一装载的隐藏保守性：前两名玩家无需为第三名的额外暴露共同支付库存成本。

最佳响应预言机对每个角色逐一枚举 70 条最短路与两种风险可行错峰装载，共 140 个偏离。筛选表保存每条路线、等待、装载、失败率和总体均值；最终表再保存六个模糊集点及 $\rho = 0.70$ 压力点上的推荐值、最佳响应值和可利用度。因入选偏离与原策略逐日消耗相同，零偏离增益不是四舍五入造成，而是配对样本上的逐样本相等。

B.4 初始装载的局部稳健性

图 14 给出第六关 $q = 180, 185, \dots, 220$ 的完整网格结果。装载增加使失败率下降，但每多带 5 箱水和 5 箱食物，初购成本增加 75 元，并挤压终值；故风险约束的可行边界比单纯追求“越安全越好”更有决策意义。入选点 $q = 185$ 在训练不利天气下失败率 0.2778%，相邻的 $q = 180$ 为 0.6778%、超过 0.5% 预算；选择对一个网格步长的扰动有清晰解释。

第六关装载敏感性：185/185是风险约束下的最小安全装载

训练集每点3000局；黑星为入选装载；失败样本终值按0计入

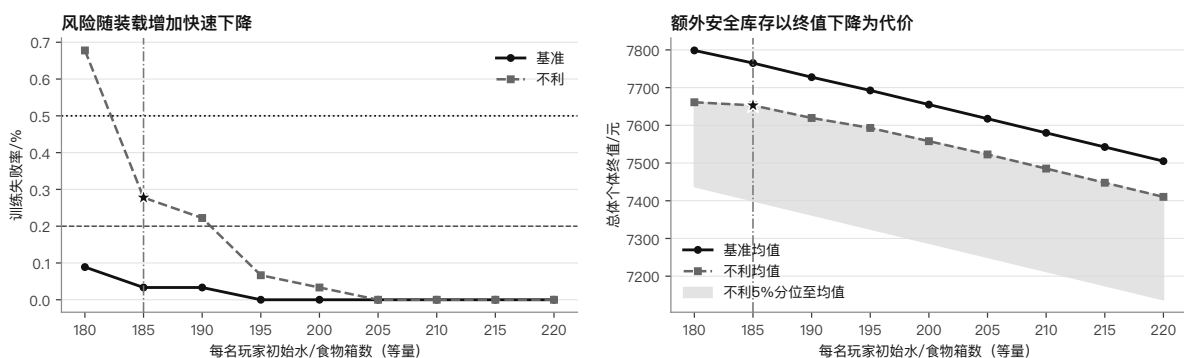


图 14 第六关分路直达策略的装载—风险—收益敏感性

B.5 角色差异与事前公平

固定三条路线时，前两种角色因路径与装载对称而样本均值相等，第三种错峰角色同时承担额外停留和更高安全库存成本；模糊集内角色差约为 155–166 元。式 (63) 证明，出发前均匀随机置换角色后，每名玩家的事前期望都等于各面板横线所示均值。该机制不改变任何一局中的路线集合，因此不会重新引入同向边拥挤。

固定路线存在角色差异，出发前等概率轮换实现事前公平
前两角色215/215，错峰角色225/230；随机置换不改变拥挤与安全性

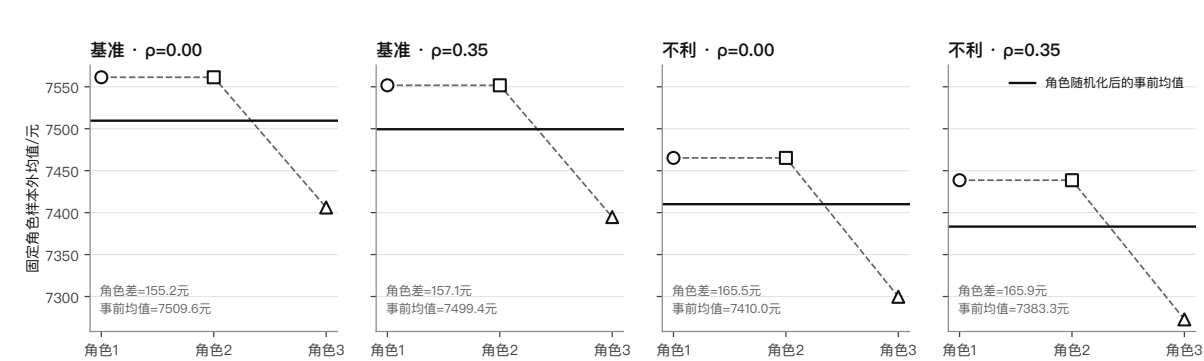


图 15 第六关固定角色收益与随机置换后的事前公平值

B.6 可证伪的复核清单

表 14 评审可独立复核的中间证据

结论	可复核文件	反证条件
确定天气全局最优	known_weather_exact.json、逐日流水	求解器非 Optimal 或任一回放约束残差非 0
第三关精确 MDP	level3_mdp_sensitivity.csv	Bellman 值与 1024 序列加权值不一致
第四关条件推荐	level4_robust_search.csv、level4_robust_comparison.csv	模糊集内任一点区间上界超预算
第四关 IID 策略缺口	level4_exact_mdp_gap.json	B5 不满足 0.2% 预算，或对偶不等式失效
第五关纯策略均衡	problem3/summary.json	任一完整最佳响应偏离增益为正
第六关风险推荐	level6_role_load_calibration.csv、level6_robust_comparison.csv	任一固定角色区间上界越界
第六关偏离稳定性	level6_oracle_screening.csv、level6_exploitability.csv	风险可行偏离的最坏分布收益增益为正

C 核心建模代码摘录

本节代码均由论文随附仓库直接读取，不是为排版另写的伪代码。为控制篇幅，只保留能够对应正文方程的核心区段；完整程序、测试和场景数据随项目一并交付。

C.1 时间扩展网络 MILP 的行动弧与流守恒

代码1对应式 (11)和(13)：按天气生成行动弧，沙暴日不生成移动变量，并以两组流约束连接相邻时间层。

Listing 1 确定天气 MILP 的行动弧与位置流构造

```

1 def _build_lp(scenario: Scenario) -> str:
2     if scenario.known_weather is None:
3         raise OptimizationError("精确求解器只接受全程天气已知的关卡")
4     days = scenario.deadline
5     binaries: list[str] = []
6     generals: list[str] = []
7     bounds: list[str] = []
8     constraints: list[str] = []
9     action_by_day: dict[int, list[tuple[str, int, int, int, int, Action]]] = {}
10    # tuple: variable, origin, destination, water consumption, food consumption, action
11
12    for day in range(days + 1):
13        for node in range(1, scenario.node_count + 1):
14            binaries.append(f"x_{day}_{node}")
15
16    for day, weather in enumerate(scenario.known_weather, start=1):
17        actions: list[tuple[str, int, int, int, int, Action]] = []
18        base_water = scenario.water.consumption[weather]
19        base_food = scenario.food.consumption[weather]
20        for node in range(1, scenario.node_count + 1):
21            if node == scenario.destination:
22                name = f"a_h_{day}"
23                binaries.append(name)
24                actions.append((name, node, node, 0, 0, Action.STAY))
25                continue
26            stay = f"a_s_{day}_{node}"
27            binaries.append(stay)
28            actions.append((stay, node, node, base_water, base_food, Action.STAY))
29            if node in scenario.mines:
30                mine = f"a_m_{day}_{node}"
31                binaries.append(mine)
32                actions.append((mine, node, node, 3 * base_water, 3 * base_food, Action.MINE))
33            if weather is not Weather.SANDSTORM:
34                for neighbour in sorted(scenario.graph[node]):
35                    walk = f"a_w_{day}_{node}_{neighbour}"
36                    binaries.append(walk)
37                    actions.append((walk, node, neighbour, 2 * base_water, 2 * base_food, Action.WALK))
38            action_by_day[day] = actions
39
40    # 起始位置与截止日终点。
41    for node in range(1, scenario.node_count + 1):
42        constraints.append(f" start_{node}: x_0_{node} = {1 if node == scenario.start else 0}")
43    constraints.append(f" finish: x_{days}_{scenario.destination} = 1")
44
45    # 每日行动的流守恒；终点只有零消耗吸收自环，不能再次离开。
46    for day in range(1, days + 1):
47        actions = action_by_day[day]
48        for node in range(1, scenario.node_count + 1):
49            outgoing = [(1, name) for name, origin, _, _, _ in actions if origin == node]
50            incoming = [(1, name) for name, _, destination, _, _ in actions if destination == node]
51            constraints.append(
52                f" out_{day}_{node}: {_expr(outgoing + [(-1, f'x_{day - 1}_{node}')] )} = 0"
53            )
54            constraints.append(
55                f" in_{day}_{node}: {_expr(incoming + [(-1, f'x_{day}_{node}')] )} = 0"
56            )

```

```

57
58 # 物资、现金与采购。
59 generals.extend(("bw_0", "bf_0", "rw_0", "rf_0"))
60 bounds.extend((" 0 <= bw_0 <= 400", " 0 <= bf_0 <= 600"))
61 constraints.append(" water_0: rw_0 - bw_0 = 0")
62 constraints.append(" food_0: rf_0 - bf_0 = 0")
63 constraints.append(
64     f" cash_0_eq: cash_0 + {scenario.water.base_price} bw_0 + {scenario.food.base_price} bf_0 = {scenario
        .initial_cash}"

```

C.2 天气揭示前后的 Bellman 递推

代码2对应式 (24)和(25)。任何正概率天气下没有可行后继，状态值即为负无穷，从实现层保证支撑集稳健可行。

Listing 2 第三关有限时域 MDP 的精确递推

```

1  def value(self, day: int, location: int, water: int, food: int) -> float:
2      return self._cached_value(day, location, water, food)
3
4  def _value_impl(self, day: int, location: int, water: int, food: int) -> float:
5      """当第 ``day`` 日天气尚未揭示时的最优未来收益（不含当前现金）。"""
6      if location == self.scenario.destination:
7          return self._salvage(water, food)
8      if day > self.scenario.deadline:
9          return NEGATIVE_INFINITY
10     expected = 0.0
11     for weather in self.weather_model.support:
12         probability = self.weather_model.probabilities[weather]
13         best, _ = self.best_after_weather(day, location, water, food, weather)
14         if best == NEGATIVE_INFINITY:
15             return NEGATIVE_INFINITY
16         expected += probability * best
17     return expected
18
19 def best_after_weather(
20     self,
21     day: int,
22     location: int,
23     water: int,
24     food: int,
25     weather: Weather,
26 ) -> tuple[float, DayDecision | None]:
27     best_value = NEGATIVE_INFINITY
28     best_decision: DayDecision | None = None
29     for decision in self._actions(location, weather):
30         transition = self._transition(location, water, food, weather, decision)
31         if transition is None:
32             continue
33         next_location, next_water, next_food, reward = transition
34         if next_location == self.scenario.destination:
35             candidate = reward + self._salvage(next_water, next_food)
36         elif day == self.scenario.deadline:
37             candidate = NEGATIVE_INFINITY
38         else:
39             future = self.value(day + 1, next_location, next_water, next_food)

```

```

40         candidate = NEGATIVE_INFINITY if future == NEGATIVE_INFINITY else reward + future
41         if candidate > best_value + 1e-9:
42             best_value = candidate
43             best_decision = decision
44         return best_value, best_decision

```

C.3 完整最佳响应动态规划

代码3对应式 (51)和(52)。状态标签同时受负重、资金和最短剩余距离剪枝，相同状态只保留更高矿山收入。

Listing 3 第五关固定对手下的完整最佳响应搜索

```

1 def best_response(scenario: Scenario, opponent_path: ActionPath | None) -> PlanEvaluation:
2     """对固定对手计划做完整有限时域动态规划，返回精确最佳响应。"""
3     if scenario.known_weather is None:
4         raise ValueError("第五关必须给定完整天气")
5     opponent_schedule = _schedule(scenario, opponent_path) if opponent_path else ()
6     # 状态值记录达到(day, location, water, food)时可获得的最大挖矿收入及路径。
7     states: dict[tuple[int, int, int], tuple[Fraction, tuple[DayDecision, ...]]] = {
8         (scenario.start, 0, 0): (Fraction(0), ())
9     }
10    terminals: list[PlanEvaluation] = []
11
12    for day in range(1, scenario.deadline + 1):
13        weather = scenario.known_weather[day - 1]
14        opponent_day = opponent_schedule[day - 1] if day <= len(opponent_schedule) else None
15        next_states: dict[tuple[int, int, int], tuple[Fraction, tuple[DayDecision, ...]]] = {}
16        for (location, used_water, used_food), (revenue, path) in states.items():
17            options = [DayDecision(Action.STAY)]
18            if weather is not Weather.SANDSTORM:
19                options.extend(DayDecision(Action.WALK, node) for node in sorted(scenario.graph[location]))
20            if location in scenario.mines:
21                options.append(DayDecision(Action.MINE))
22
23            for decision in options:
24                destination = int(decision.destination) if decision.action is Action.WALK else location
25                dw, df, daily_revenue = _daily_effect(
26                    scenario, weather, location, decision, opponent_day
27                )
28                water = used_water + dw
29                food = used_food + df
30                if water * scenario.water.mass + food * scenario.food.mass > scenario.capacity:
31                    continue
32                if water * scenario.water.base_price + food * scenario.food.base_price > scenario.initial_cash:
33                    continue
34                decisions = path + (decision,)
35                total_revenue = revenue + daily_revenue
36                if destination == scenario.destination:
37                    purchase_cost = (
38                        water * scenario.water.base_price + food * scenario.food.base_price
39                    )
40                    terminals.append(
41                        PlanEvaluation(
42                            ActionPath(decisions),

```

```

43         water,
44         food,
45         day,
46         total_revenue,
47         Fraction(scenario.initial_cash - purchase_cost) + total_revenue,
48     )
49 )
50     continue
51     remaining = scenario.deadline - day
52     if shortest_distance(scenario, destination, scenario.destination) > remaining:
53         continue
54     key = (destination, water, food)
55     incumbent = next_states.get(key)
56     if incumbent is None or total_revenue > incumbent[0]:
57         next_states[key] = (total_revenue, decisions)
58     states = next_states
59
60 if not terminals:
61     raise RuntimeError("不存在可行响应")
62 return max(
63     terminals,
64     key=lambda item: (
65         item.final_value,
66         -item.arrival_day,
67         tuple((d.action.value, d.destination or 0) for d in item.path.decisions),
68     ),
69 )

```

C.4 固定装载 MDP 与风险对偶

代码4实现式 (41)和(42)的共同状态转移。失败分支严格返回 0，成功终点才结算初始剩余现金、矿山收益和半价回收价值；因此拉格朗日惩罚不会把失败路径误计为成功收益。

Listing 4 第四关固定装载精确 MDP、失败终值与拉格朗日评分

```

1  动作，但将装载固定为给定值。这样状态只含 ``(天数，位置，水，食物)``，
2  可精确求解，用于量化阈值规则在相同装载下的动态决策缺口。
3  """
4
5  def __init__(
6      self,
7      scenario: Scenario,
8      weather_model: IIDWeatherModel,
9      initial_water: int,
10     initial_food: int,
11     failure_penalty: float = 0.0,
12 ) -> None:
13     if scenario.level != 4:
14         raise ValueError("Level4FixedLoadMDP 仅适用于第四关")
15     if scenario.known_weather is not None:
16         raise ValueError("精确基准需要未来天气未知的场景")
17     if initial_water < 0 or initial_food < 0:
18         raise ValueError("初始装载必须非负")
19     if initial_water * scenario.water.mass + initial_food * scenario.food.mass > scenario.capacity:

```

```

20         raise ValueError("初始装载超过负重上限")
21     if initial_water * scenario.water.base_price + initial_food * scenario.food.base_price > scenario.
        initial_cash:
22         raise ValueError("初始装载超过初始资金")
23     if failure_penalty < 0:
24         raise ValueError("失败惩罚系数必须非负")
25     self.scenario = scenario
26     self.weather_model = weather_model
27     self.initial_water = initial_water
28     self.initial_food = initial_food
29     self.failure_penalty = failure_penalty
30     self._optimal_value = lru_cache(maxsize=None)(self._optimal_value_impl)
31     self._optimal_after_weather = lru_cache(maxsize=None)(self._optimal_after_weather_impl)
32     self._threshold_value = lru_cache(maxsize=None)(self._threshold_value_impl)
33
34     def _salvage(self, water: int, food: int) -> float:
35         return 0.5 * self.scenario.water.base_price * water + 0.5 * self.scenario.food.base_price * food
36
37     def _initial_cash(self) -> float:
38         return float(
39             self.scenario.initial_cash
40             - self.initial_water * self.scenario.water.base_price
41             - self.initial_food * self.scenario.food.base_price
42         )
43
44     def _actions(self, location: int, weather: Weather) -> tuple[DayDecision, ...]:
45         actions = [DayDecision(Action.STAY)]
46         if weather is not Weather.SANDSTORM:
47             actions.extend(DayDecision(Action.WALK, neighbour) for neighbour in sorted(self.scenario.graph[
                location]))
48         if location in self.scenario.mines:
49             actions.append(DayDecision(Action.MINE))
50         return tuple(actions)
51
52     def _transition(
53         self,
54         location: int,
55         water: int,
56         food: int,
57         weather: Weather,
58         decision: DayDecision,
59     ) -> tuple[int, int, int, float] | None:
60         if decision.action is Action.WALK:
61             if weather is Weather.SANDSTORM or decision.destination not in self.scenario.graph[location]:
62                 return None
63             next_location = int(decision.destination)
64             factor, reward = 2, 0.0
65         elif decision.action is Action.STAY:
66             next_location, factor, reward = location, 1, 0.0
67         elif decision.action is Action.MINE:
68             if location not in self.scenario.mines:
69                 return None
70             next_location, factor, reward = location, 3, float(self.scenario.mine_income)
71         else:
72             return None
73         next_water = water - factor * self.scenario.water.consumption[weather]
74         next_food = food - factor * self.scenario.food.consumption[weather]

```

```

75     if next_water < 0 or next_food < 0:
76         return None
77     if next_location != self.scenario.destination and (next_water == 0 or next_food == 0):
78         return None
79     return next_location, next_water, next_food, reward
80
81     def _terminal_or_future(
82         self,
83         day: int,
84         transition: tuple[int, int, int, float] | None,
85         value_function,
86         extra_state: tuple[int, int] = (),
87     ) -> BenchmarkOutcome:
88         if transition is None:
89             return BenchmarkOutcome(0.0, 1.0)
90         location, water, food, reward = transition
91         if location == self.scenario.destination:
92             # 初始剩余现金只在成功到达时才计入终值；失败分支必须严格为 0。
93             return BenchmarkOutcome(self._initial_cash() + reward + self._salvage(water, food), 0.0)
94         if day == self.scenario.deadline:
95             return BenchmarkOutcome(0.0, 1.0)
96         future = value_function(day + 1, location, water, food, *extra_state)
97         return BenchmarkOutcome(reward + future.expected_final_value, future.failure_probability)
98
99     def _optimal_value_impl(self, day: int, location: int, water: int, food: int) -> BenchmarkOutcome:
100         expected_value = 0.0
101         failure_probability = 0.0
102         for weather in self.weather_model.support:
103             probability = self.weather_model.probabilities[weather]
104             outcome = self._optimal_after_weather(day, location, water, food, weather)
105             expected_value += probability * outcome.expected_final_value
106             failure_probability += probability * outcome.failure_probability
107         return BenchmarkOutcome(expected_value, failure_probability)
108
109     def _optimal_after_weather_impl(
110         self,
111         day: int,

```

C.5 多人联合分组与逐日结算

代码5先按同向有向边和同矿建立分组，再由组大小计算消耗倍数和收益分成，对应式 (45)、(46)和(48)。

Listing 5 第六关多人拥挤分组、资源扣除与终值结算

```

1     states = _purchase_groups(scenario, states, decisions, before=True)
2
3     walk_groups: dict[tuple[int, int], list[int]] = {}
4     mine_groups: dict[int, list[int]] = {}
5     for index, (state, decision) in enumerate(zip(states, decisions)):
6         if state.failed or state.arrival_day is not None:
7             if decision is not None:
8                 raise SimulationError("失败或到达玩家仍在行动")
9             continue
10        if decision is None:
11            raise SimulationError("活跃玩家缺少行动")

```

```

12     if decision.action is Action.WALK:
13         if today is Weather.SANDSTORM:
14             raise SimulationError("沙暴日不能行走")
15         if decision.destination not in scenario.graph[state.location]:
16             raise SimulationError("多人策略移动不相邻")
17         walk_groups.setdefault((state.location, int(decision.destination)), []).append(index)
18     elif decision.action is Action.MINE:
19         if state.location not in scenario.mines:
20             raise SimulationError("多人策略在非矿山挖矿")
21         mine_groups.setdefault(state.location, []).append(index)
22     elif decision.action is not Action.STAY:
23         raise SimulationError("多人行动类型无效")
24
25 next_states = list(states)
26 for index, (state, decision) in enumerate(zip(states, decisions)):
27     if decision is None:
28         continue
29     cash = state.cash
30     location = state.location
31     if decision.action is Action.WALK:
32         group_size = len(walk_groups[(state.location, int(decision.destination))])
33         factor = 2 * group_size if group_size >= 2 else 2
34         location = int(decision.destination)
35     elif decision.action is Action.MINE:
36         group_size = len(mine_groups[state.location])
37         factor = 3
38         cash += Fraction(scenario.mine_income, group_size)
39     else:
40         factor = 1
41     water = state.water - factor * scenario.water.consumption[today]
42     food = state.food - factor * scenario.food.consumption[today]
43     failed = water < 0 or food < 0 or (
44         location != scenario.destination and (water == 0 or food == 0)
45     )
46     arrival_day = day if location == scenario.destination and not failed else None
47     next_states[index] = PlayerState(
48         location, cash, water, food, arrival_day, failed
49     )
50     states = _purchase_groups(scenario, next_states, decisions, before=False)
51     records.append(MultiDayRecord(day, today, tuple(states), decisions))
52
53 final_values: list[Fraction | None] = []
54 for state in states:
55     if state.arrival_day is None or state.failed:
56         final_values.append(None)
57     else:
58         final_values.append(
59             state.cash
60             + Fraction(scenario.water.base_price * state.water, 2)
61             + Fraction(scenario.food.base_price * state.food, 2)
62         )

```

C.6 Wilson 失败率区间

代码6直接实现式 (65)和(66)，并用于第四、六关所有样本外失败率区间。

Listing 6 Wilson 得分区间的数值实现

```

1 def _quantile(values: list[float], probability: float) -> float:
2     ordered = sorted(values)
3     index = max(0, min(len(ordered) - 1, int(probability * (len(ordered) - 1))))
4     return ordered[index]
5
6
7 def _wilson_interval(successes: int, samples: int, z: float = 1.959963984540054) -> tuple[float, float]:
8     """二项比例 Wilson 区间；这里的 ``successes`` 可表示失败次数。"""
9     if samples <= 0:
10         raise ValueError("样本数必须为正")
11     proportion = successes / samples
12     denominator = 1 + z * z / samples
13     centre = (proportion + z * z / (2 * samples)) / denominator
14     radius = z * math.sqrt(
15         proportion * (1 - proportion) / samples + z * z / (4 * samples * samples)
16     ) / denominator
17     return max(0.0, centre - radius), min(1.0, centre + radius)

```

C.7 保持平稳边际的 Markov 天气生成器

代码7实现式 (35)。构造函数逐行验证初始分布与转移矩阵，`from_stationary` 再以 $\rho I + (1 - \rho)1p^\top$ 生成转移概率；采样时只使用当前天气，不访问未来状态。

Listing 7 保持平稳边际的 Markov 天气模型与序列采样

```

1 @dataclass(frozen=True)
2 class MarkovWeatherModel:
3     """有限状态一阶 Markov 天气模型。
4
5     from_stationary 使用  $P_{\rho} = \rho I + (1 - \rho) 1 p^\top$  构造转移矩阵，
6     因此在改变连续天气聚集程度的同时严格保持长期边际分布  $p$  不变。
7     """
8
9     initial_probabilities: Mapping[Weather, float]
10    transition_probabilities: Mapping[Weather, Mapping[Weather, float]]
11    persistence: float | None = None
12
13    def __post_init__(self) -> None:
14        initial = {
15            weather: float(self.initial_probabilities.get(weather, 0.0))
16            for weather in Weather
17        }
18        if any(value < 0 for value in initial.values()):
19            raise ValueError("初始天气概率不能为负")
20        if abs(sum(initial.values()) - 1.0) > 1e-9:
21            raise ValueError("初始天气概率之和必须为 1")
22
23        transition: dict[Weather, dict[Weather, float]] = {}
24        for origin in Weather:
25            row = self.transition_probabilities.get(origin)
26            if row is None:
27                raise ValueError(f"缺少天气 {origin.value} 的转移概率")
28            row = {destination: float(row.get(destination, 0.0)) for destination in Weather}
29            if any(value < 0 for value in row.values()):
30                raise ValueError("天气转移概率不能为负")

```



```

31         if abs(sum(row.values()) - 1.0) > 1e-9:
32             raise ValueError(f"天气 {origin.value} 的转移概率之和必须为 1")
33         transition[origin] = row
34
35         object.__setattr__(self, "initial_probabilities", initial)
36         object.__setattr__(self, "transition_probabilities", transition)
37
38     @classmethod
39     def from_stationary(
40         cls,
41         probabilities: Mapping[Weather, float],
42         persistence: float,
43     ) -> "MarkovWeatherModel":
44         if not 0.0 <= persistence < 1.0:
45             raise ValueError("持续性参数必须满足 0 <= rho < 1")
46         stationary = {weather: float(probabilities.get(weather, 0.0)) for weather in Weather}
47         if any(value < 0 for value in stationary.values()):
48             raise ValueError("平稳天气概率不能为负")
49         if abs(sum(stationary.values()) - 1.0) > 1e-9:
50             raise ValueError("平稳天气概率之和必须为 1")
51         transition = {
52             origin: {
53                 destination: (
54                     persistence * float(origin is destination)
55                     + (1.0 - persistence) * stationary[destination]
56                 )
57             for destination in Weather
58         }
59         for origin in Weather
60     }
61     return cls(stationary, transition, persistence)
62
63     @property
64     def support(self) -> tuple[Weather, ...]:
65         return tuple(
66             weather
67             for weather in Weather
68             if self.initial_probabilities[weather] > 0
69         )
70
71     def sample(self, days: int, rng: random.Random) -> tuple[Weather, ...]:
72         if days < 0:
73             raise ValueError("天气天数不能为负")
74         if days == 0:
75             return ()
76         all_weather = tuple(Weather)
77         first = rng.choices(
78             all_weather,
79             weights=[self.initial_probabilities[weather] for weather in all_weather],
80             k=1,
81         )[0]
82         sequence = [first]
83         for _ in range(1, days):
84             previous = sequence[-1]
85             sequence.append(
86                 rng.choices(
87                     all_weather,

```

```

88         weights=[
89             self.transition_probabilities[previous][weather]
90             for weather in all_weather
91         ],
92         k=1,
93     )[0]
94 )
95 return tuple(sequence)

```

C.8 最短路偏离类与异质角色策略

代码8先按到终点的 BFS 距离，只沿距离减 1 的边深搜，从而完整枚举 70 条最短路；随后将每名玩家的路线、装载和错峰天数封装为可单边替换的策略剖面。这一结构保证最佳响应实验改动玩家 i 时， π_{-i} 保持逐字节一致。

Listing 8 第六关最短路枚举、角色策略与单边偏离构造

```

1 def _distances_to(scenario: Scenario, destination: int) -> dict[int, int]:
2     distances = {destination: 0}
3     queue = deque([destination])
4     while queue:
5         node = queue.popleft()
6         for neighbour in scenario.graph[node]:
7             if neighbour not in distances:
8                 distances[neighbour] = distances[node] + 1
9                 queue.append(neighbour)
10    return distances
11
12
13 def enumerate_shortest_routes(scenario: Scenario) -> tuple[tuple[int, ...], ...]:
14     """枚举起点到终点的全部最短路，顺序固定以保证实验可复现。"""
15     distances = _distances_to(scenario, scenario.destination)
16     routes: list[tuple[int, ...]] = []
17
18     def visit(node: int, prefix: tuple[int, ...]) -> None:
19         if node == scenario.destination:
20             routes.append(prefix)
21             return
22         for neighbour in sorted(scenario.graph[node]):
23             if distances.get(neighbour) == distances[node] - 1:
24                 visit(neighbour, prefix + (neighbour,))
25
26     visit(scenario.start, (scenario.start,))
27     return tuple(routes)
28
29
30 def _route_step(route: tuple[int, ...], location: int) -> int:
31     try:
32         index = route.index(location)
33     except ValueError as exc:
34         raise RuntimeError(f"位置 {location} 不在偏离路线中") from exc
35     if index + 1 >= len(route):
36         return location
37     return route[index + 1]
38

```

```

39
40 @dataclass(frozen=True)
41 class RouteProfilePolicy(JointOnlinePolicy):
42     """第六关的异质最短路—装载—错峰策略剖面。"""
43
44     routes: tuple[tuple[int, ...], ...]
45     initial_loads: tuple[tuple[int, int], ...]
46     voluntary_waits: tuple[int, ...]
47
48     def __post_init__(self) -> None:
49         if not (
50             len(self.routes) == len(self.initial_loads) == len(self.voluntary_waits)
51         ):
52             raise ValueError("路线、装载与等待向量长度必须一致")
53         if any(wait < 0 for wait in self.voluntary_waits):
54             raise ValueError("自愿等待天数不能为负")
55
56     def initial_purchases(self, scenario: Scenario) -> tuple[tuple[int, int], ...]:
57         if len(self.initial_loads) != scenario.player_count:
58             raise ValueError("策略人数与场景人数不一致")
59         return self.initial_loads
60
61     def decide(
62         self,
63         scenario: Scenario,
64         history: tuple[MultiDayRecord, ...],
65         weather: Weather,
66     ) -> tuple[DayDecision | None, ...]:
67         decisions: list[DayDecision | None] = []
68         for player, state in enumerate(history[-1].states):
69             if state.failed or state.arrival_day is not None:
70                 decisions.append(None)
71                 continue
72             waits_used = sum(
73                 record.decisions[player] is not None
74                 and record.decisions[player].action is Action.STAY
75                 and record.weather is not Weather.SANDSTORM
76                 for record in history[1:]
77             )
78             if (
79                 state.location == scenario.start
80                 and waits_used < self.voluntary_waits[player]
81                 and weather is not Weather.SANDSTORM
82             ):
83                 decisions.append(DayDecision(Action.STAY))
84             elif weather is Weather.SANDSTORM:
85                 decisions.append(DayDecision(Action.STAY))
86             else:
87                 decisions.append(
88                     DayDecision(
89                         Action.WALK,
90                         _route_step(self.routes[player], state.location),
91                     )
92                 )
93         return tuple(decisions)
94
95     def unilateral_deviation(

```

```

96     self,
97     player: int,
98     *,
99     route: tuple[int, ...] | None = None,
100     initial_water: int | None = None,
101     initial_food: int | None = None,
102     voluntary_wait: int | None = None,
103 ) -> "RouteProfilePolicy":
104     routes = list(self.routes)
105     loads = list(self.initial_loads)
106     waits = list(self.voluntary_waits)
107     if route is not None:
108         routes[player] = route
109     water, food = loads[player]
110     loads[player] = (
111         water if initial_water is None else initial_water,
112         food if initial_food is None else initial_food,
113     )
114     if voluntary_wait is not None:
115         waits[player] = voluntary_wait
116     return replace(
117         self,
118         routes=tuple(routes),
119         initial_loads=tuple.loads),
120         voluntary_waits=tuple(waits),
121     )

```

C.9 二维装载校准与预言机筛选

代码9对三种错峰角色枚举二维装载，并在模糊集所有点上以 Wilson 上界判定可行；代码10则对每名玩家完整扫描风险校准后的路线—错峰偏离类。

Listing 9 角色条件装载的分布鲁棒校准

```

1  def calibrate_direct_loads(
2      samples: int,
3  ) -> tuple[dict[int, tuple[int, int]], list[dict]]:
4      suite = _sample_suite(6, IN_SET_RHOS, samples, SEED + 300)
5      rows: list[dict] = []
6      selected: dict[int, tuple[int, int]] = {}
7      for wait in (0, 1, 2):
8          requirements = {
9              key: [_direct_requirements(6, sequence, wait) for sequence in sequences]
10             for key, sequences in suite.items()
11          }
12      wait_rows: list[dict] = []
13      for water in range(180, 241, 5):
14          for food in range(180, 241, 5):
15              if 3 * water + 2 * food > get_scenario(6).capacity:
16                  continue
17              feasible = True
18              worst_mean = float("inf")
19              worst_upper = 0.0
20              for (name, _rho), values in requirements.items():
21                  failures = sum(w > water or f > food for w, f in values)
22                  upper = _wilson(failures, len(values))[1]

```

```

23         feasible &= upper <= RISK_BUDGETS[name]
24         worst_upper = max(worst_upper, upper)
25         worst_mean = min(
26             worst_mean,
27             mean(_requirement_value(water, food, value) for value in values),
28         )
29         row = {
30             "voluntary_waits": wait,
31             "initial_water": water,
32             "initial_food": food,
33             "samples_per_model": samples,
34             "feasible": feasible,
35             "worst_wilson_high": worst_upper,
36             "worst_mean_value": worst_mean,
37         }
38         wait_rows.append(row)
39         rows.append(row)
40     feasible_rows = [row for row in wait_rows if row["feasible"]]
41     if feasible_rows:
42         best = max(feasible_rows, key=lambda row: row["worst_mean_value"])
43         selected[wait] = (int(best["initial_water"]), int(best["initial_food"]))
44     return selected, rows
45
46
47 def recommended_profile(loads_by_wait: dict[int, tuple[int, int]]) -> RouteProfilePolicy:
48     direct = CoordinatedDirectPolicy()
49     return RouteProfilePolicy(
50         routes=direct.routes,
51         initial_loads=(loads_by_wait[0], loads_by_wait[0], loads_by_wait[1]),
52         voluntary_waits=(0, 0, 1),
53     )

```

Listing 10 受约束最佳响应预言机的完整候选筛选

```

1 def best_response_oracle(
2     profile: RouteProfilePolicy,
3     loads_by_wait: dict[int, tuple[int, int]],
4     training_samples: int,
5     test_samples: int,
6 ) -> tuple[list[dict], list[dict]]:
7     scenario = get_scenario(6)
8     routes = enumerate_shortest_routes(scenario)
9     worst_model = MarkovWeatherModel.from_stationary(MODELS["不利"].probabilities, 0.35)
10    rng = random.Random(SEED + 500)
11    training_weather = [
12        worst_model.sample(scenario.deadline, rng) for _ in range(training_samples)
13    ]
14    screening: list[dict] = []
15    selected: dict[int, RouteProfilePolicy] = {}
16    for player in range(scenario.player_count):
17        player_rows: list[tuple[dict, RouteProfilePolicy]] = []
18        for route_index, route in enumerate(routes):
19            for wait, (water, food) in sorted(loads_by_wait.items()):
20                candidate = profile.unilateral_deviation(
21                    player,
22                    route=route,
23                    initial_water=water,

```

```

24         initial_food=food,
25         voluntary_wait=wait,
26     )
27     result = _evaluate_role(candidate, player, training_weather)
28     row = {
29         "player": player + 1,
30         "route_index": route_index,
31         "route": "-".join(map(str, route)),
32         "voluntary_waits": wait,
33         "initial_water": water,
34         "initial_food": food,
35         **result,
36         "training_risk_budget": RISK_BUDGETS["不利"],
37         "training_feasible": result["failure_rate"]
38         <= RISK_BUDGETS["不利"],
39     }
40     screening.append(row)
41     player_rows.append((row, candidate))
42     feasible = [item for item in player_rows if item[0]["training_feasible"]]
43     selected[player] = max(feasible, key=lambda item: item[0]["mean_value"])[1]

```

C.10 代码—公式—输出追踪关系

表 15 建模证据的三向追踪

数学对象	实现入口	主要审计输出
时空网络 MILP	optimizer.py	求解状态、目标值、逐日位置和资源账本
有限时域 MDP	level3_mdp.py	状态数、初购、Bellman 值、全序列误差
风险阈值策略	level4_policies.py	候选前沿、失败率、Wilson 区间
固定装载精确基准	level4_mdp.py	风险可行下界、拉格朗日对偶上界
最佳响应博弈	level5_game.py	均衡路径、两方偏离增益
多人联合转移	multiplayer.py	共享边/矿山玩家日、角色均值、失败率
Markov 天气	weather.py	平稳边际、转移概率、连续沙暴统计
分布鲁棒校准	analyze_robustness.py	模糊集压力曲线、角色装载、偏离可利用度
统计评估	evaluation.py	均值、分位数、均值区间、Wilson 区间